SIPTestWin

Documentacion tecnica: codigo fuente comentado y guia de uso

Herramienta de pruebas SIP (RFC 3261) para Windows, inspirada en SIPp. Version 0.1.0.

Uso autorizado unicamente. Solo contra sistemas propios o con permiso explicito. Incluye lista blanca de destinos, limites, modo simulacion por defecto, confirmacion, redaccion de credenciales y auditoria.

Contenido

1. Guia de uso con ejemplos basicos
2. Codigo fuente comentado, modulo por modulo
   - src/siptestwin/__init__.py
   - src/siptestwin/sip_message.py
   - src/siptestwin/transaction.py
   - src/siptestwin/dialog.py
   - src/siptestwin/transport/__init__.py
   - src/siptestwin/transport/base.py
   - src/siptestwin/transport/udp.py
   - src/siptestwin/transport/tcp.py
   - src/siptestwin/transport/factory.py
   - src/siptestwin/authentication/__init__.py
   - src/siptestwin/authentication/digest.py
   - src/siptestwin/config.py
   - src/siptestwin/safety.py
   - src/siptestwin/scenario_parser.py
   - src/siptestwin/engine.py
   - src/siptestwin/metrics.py
   - src/siptestwin/reporting.py
   - src/siptestwin/capture.py
   - src/siptestwin/netutils.py
   - src/siptestwin/security_tests/__init__.py
   - src/siptestwin/security_tests/protocol.py
   - src/siptestwin/security_tests/malformed.py
   - src/siptestwin/security_tests/regression.py
   - src/siptestwin/testserver.py
   - src/siptestwin/cli.py
   - src/siptestwin/__main__.py

1. Guia de uso con ejemplos basicos

Instalacion

Crea el entorno virtual fuera de OneDrive e instala el paquete:

```
py -m venv C:\Users\tu_usuario\.venvs\siptestwin
C:\Users\tu_usuario\.venvs\siptestwin\Scripts\Activate.ps1
pip install -e .
siptestwin version
siptestwin doctor
```

Es una herramienta de linea de comandos: ejecutala desde PowerShell, no con doble clic.

version / doctor

Muestra la version y diagnostica el entorno (Python, dependencias, enlace UDP, avisos de firewall):

```
siptestwin version
siptestwin doctor
```

interfaces (elegir la IP local)

Lista las IPv4 locales disponibles para usar como IP de origen (Via/Contact). Elige una con --local-host o fijala en execution.local_host.

```
siptestwin interfaces
siptestwin run -c config\example.yaml --local-host 192.168.1.26 ...
```

validate-config / validate-scenario

Valida la configuracion y el escenario antes de ejecutar:

```
siptestwin validate-config   --config config\example.yaml
siptestwin validate-scenario --file scenarios\basic_call.xml
```

testserver (servidor de prueba local)

Arranca un servidor SIP de prueba en loopback para validar sin sistemas externos:

```
siptestwin testserver --port 55060
siptestwin testserver --port 55060 --require-auth
```

run (simulacion y ejecucion activa)

Por defecto SIMULA (no transmite). Con --execute transmite, pide confirmacion y exige que el destino este autorizado:

```
# simulacion (no envia paquetes)
siptestwin run -c config\example.yaml --scenario scenarios\options.xml ^
  --host 127.0.0.1 --port 55060

# ejecucion real (transmite)
siptestwin run -c config\example.yaml --scenario scenarios\basic_call.xml ^
  --host 127.0.0.1 --port 55060 --calls 3 --execute
```

Trazar / capturar mensajes SIP

Registra cada mensaje en consola, en un .trace.log y/o en un .pcap abrible en Wireshark (sin tshark):

```
siptestwin run -c config\example.yaml --scenario scenarios\basic_call.xml ^
  --host 127.0.0.1 --port 55060 --execute ^
  --trace-console --pcap --trace-file results\mytrace

# genera results\mytrace.trace.log y results\mytrace.pcap
```

Parametrizar el INVITE y los headers

Por escenario, dentro de <send request="INVITE">:

```
<send request="INVITE">
  <header name="Subject">INVITE personalizado</header>
  <header name="X-Test-Id">[call_id]</header>
  <header name="Allow" action="remove"/>
  <body content-type="application/sdp"><![CDATA[
v=0
o=siptestwin 0 0 IN IP4 [local_ip]
s=Custom Session
c=IN IP4 [local_ip]
t=0 0
m=audio 40000 RTP/AVP 0
]]></body>
</send>
```

O de forma global en el YAML (seccion message):

```
message:
  extra_headers:
    Subject: "SIPTestWin test"
  remove_headers: ["User-Agent"]
  invite:
    extra_headers: { Priority: "urgent" }
    sdp: |
      v=0
      o=siptestwin 0 0 IN IP4 [local_ip]
      ...
```

report (resumir ejecuciones previas)

```
siptestwin report --input results\
```

Codigos de salida

0 exito • 1 cancelado • 2 config/escenario invalido • 3 destino no autorizado • 4 ejecucion con llamadas fallidas.

2. Codigo fuente comentado, modulo por modulo

Cada modulo se muestra con numeracion de linea. Las lineas incluyen los docstrings y comentarios del propio codigo, que explican el porque de cada parte.

src/siptestwin/__init__.py

Metadatos del paquete y version. *(11 lineas)*

```
"""SIPTestWin - SIP testing toolkit for Windows (SIPp-inspired).

A configurable SIP test generator / validator implementing an explicit SIP
transaction and dialog state machine on top of UDP/TCP transports. Intended
for authorized interoperability, security and controlled-load testing of SIP
servers that you own or have explicit written permission to test.
"""

__version__ = "0.1.0"

__all__ = ["__version__"]
```

src/siptestwin/sip_message.py

Nucleo del protocolo: modelo de mensaje SIP (RFC 3261). Parsea y construye peticiones/respuestas, gestiona headers multi-valor, genera Call-ID, tags y branches, y produce cuerpos SDP. *(445 lineas)*

```
"""SIP message model: parsing and construction (RFC 3261).

This module provides a transport-agnostic representation of SIP requests and
responses, a tolerant parser, and helpers to build well-formed messages with
correctly generated Call-ID, tags, branch parameters and CSeq values.

Only the subset of RFC 3261 needed for OPTIONS / REGISTER / INVITE / ACK /
CANCEL / BYE testing is implemented, but the structure is general.
"""

from __future__ import annotations

import os
import secrets
import time
from dataclasses import dataclass, field
from typing import Iterable

CRLF = "\r\n"
MAGIC_COOKIE = "z9hG4bK"  # RFC 3261 Via branch magic cookie
MAX_MESSAGE_BYTES = 64 * 1024  # hard cap to avoid unbounded parsing

# Methods we understand. Unknown methods are still parseable but flagged.
KNOWN_METHODS = {
    "INVITE", "ACK", "BYE", "CANCEL", "OPTIONS", "REGISTER",
    "INFO", "PRACK", "UPDATE", "SUBSCRIBE", "NOTIFY", "REFER", "MESSAGE",
}

# Compact header forms -> canonical names (RFC 3261 section 20, 7.3.3).
_COMPACT = {
    "i": "Call-ID", "m": "Contact", "e": "Content-Encoding",
    "l": "Content-Length", "c": "Content-Type", "f": "From",
    "s": "Subject", "k": "Supported", "t": "To", "v": "Via",
    "r": "Refer-To", "b": "Referred-By", "o": "Event",
}


class SIPParseError(ValueError):
    """Raised when a byte string cannot be parsed as a SIP message."""


def canonical_header(name: str) -> str:
    """Return the canonical, case-correct header name."""
    key = name.strip()
    low = key.lower()
    if len(low) == 1 and low in _COMPACT:
        return _COMPACT[low]
    # Title-case each hyphen-separated token but keep known special spellings.
    special = {
```

```
        "call-id": "Call-ID", "cseq": "CSeq", "www-authenticate": "WWW-Authenticate",
        "mime-version": "MIME-Version", "sip-etag": "SIP-ETag", "sip-if-match": "SIP-If-Match",
    }
    if low in special:
        return special[low]
    return "-".join(part[:1].upper() + part[1:] for part in low.split("-"))


class Headers:
    """Ordered, case-insensitive multi-value header container.

    Preserves insertion order and allows multiple values for the same header
    name (e.g. multiple Via lines), which SIP requires.
    """

    def __init__(self) -> None:
        self._items: list[tuple[str, str]] = []

    def add(self, name: str, value: str) -> None:
        self._items.append((canonical_header(name), str(value)))

    def set(self, name: str, value: str) -> None:
        """Replace all existing values for ``name`` with a single value."""
        cname = canonical_header(name)
        self._items = [(n, v) for (n, v) in self._items if n != cname]
        self._items.append((cname, str(value)))

    def get(self, name: str, default: str | None = None) -> str | None:
        """Return the first value for ``name`` (top-most)."""
        cname = canonical_header(name)
        for n, v in self._items:
            if n == cname:
                return v
        return default

    def get_all(self, name: str) -> list[str]:
        cname = canonical_header(name)
        return [v for n, v in self._items if n == cname]

    def remove(self, name: str) -> None:
        cname = canonical_header(name)
        self._items = [(n, v) for (n, v) in self._items if n != cname]

    def names(self) -> list[str]:
        seen: list[str] = []
        for n, _ in self._items:
            if n not in seen:
                seen.append(n)
        return seen

    def items(self) -> list[tuple[str, str]]:
        return list(self._items)

    def __contains__(self, name: str) -> bool:
        return self.get(name) is not None

    def __iter__(self) -> Iterable[tuple[str, str]]:
        return iter(self._items)


@dataclass
class SIPMessage:
    """A SIP request or response."""

    # For requests:
    method: str | None = None
    request_uri: str | None = None
    # For responses:
    status_code: int | None = None
    reason_phrase: str | None = None
    # Common:
    version: str = "SIP/2.0"
    headers: Headers = field(default_factory=Headers)
    body: bytes = b""

    # ----- classification ---------------------------------------------------
    @property
    def is_request(self) -> bool:
        return self.method is not None

    @property
    def is_response(self) -> bool:
        return self.status_code is not None

```

```
    @property
    def is_provisional(self) -> bool:
        return self.is_response and 100 <= (self.status_code or 0) < 200

    @property
    def is_final(self) -> bool:
        return self.is_response and (self.status_code or 0) >= 200

    # ----- convenience header accessors ---------------------------------
    @property
    def call_id(self) -> str | None:
        return self.headers.get("Call-ID")

    @property
    def cseq_number(self) -> int | None:
        raw = self.headers.get("CSeq")
        if not raw:
            return None
        try:
            return int(raw.split()[0])
        except (ValueError, IndexError):
            return None

    @property
    def cseq_method(self) -> str | None:
        raw = self.headers.get("CSeq")
        if not raw:
            return None
        parts = raw.split()
        return parts[1].upper() if len(parts) > 1 else None

    def top_via_branch(self) -> str | None:
        via = self.headers.get("Via")
        if not via:
            return None
        return _extract_param(via, "branch")

    def from_tag(self) -> str | None:
        return _extract_param(self.headers.get("From", ""), "tag")

    def to_tag(self) -> str | None:
        return _extract_param(self.headers.get("To", ""), "tag")

    # ----- serialization ------------------------------------------------
    def serialize(self) -> bytes:
        """Return the on-the-wire byte representation.

        Content-Length is always recomputed to match the body so that
        manually assembled messages stay consistent.
        """
        if self.is_request:
            start = f"{self.method} {self.request_uri} {self.version}"
        elif self.is_response:
            start = f"{self.version} {self.status_code} {self.reason_phrase or ''}".rstrip()
        else:
            raise SIPParseError("message is neither a request nor a response")

        self.headers.set("Content-Length", str(len(self.body)))
        lines = [start]
        for name, value in self.headers.items():
            lines.append(f"{name}: {value}")
        head = CRLF.join(lines) + CRLF + CRLF
        return head.encode("utf-8") + self.body

    def __str__(self) -> str:  # pragma: no cover - debugging aid
        return self.serialize().decode("utf-8", errors="replace")


# ---------------------------------------------------------------------------
# Parsing
# ---------------------------------------------------------------------------

def _split_head_body(data: bytes) -> tuple[bytes, bytes]:
    idx = data.find(b"\r\n\r\n")
    if idx != -1:
        return data[:idx], data[idx + 4:]
    idx = data.find(b"\n\n")  # tolerate LF-only separators
    if idx != -1:
        return data[:idx], data[idx + 2:]
    return data, b""


def parse(data: bytes, *, strict: bool = False) -> SIPMessage:
    """Parse ``data`` into a :class:`SIPMessage`.
```

```
    With ``strict=False`` (default) the parser tolerates LF-only line endings
    and does not require Content-Length. With ``strict=True`` it rejects such
    deviations, which is useful for protocol-validation security tests.
    """
    if len(data) > MAX_MESSAGE_BYTES:
        raise SIPParseError(f"message exceeds {MAX_MESSAGE_BYTES} bytes")
    if not data.strip():
        raise SIPParseError("empty message")

    head, body = _split_head_body(data)
    text = head.decode("utf-8", errors="replace")
    # Unfold continuation lines (a line starting with SP/HTAB continues prev).
    raw_lines = text.split("\r\n") if "\r\n" in text else text.split("\n")
    lines: list[str] = []
    for ln in raw_lines:
        if ln[:1] in (" ", "\t") and lines:
            lines[-1] += " " + ln.strip()
        else:
            lines.append(ln)
    if not lines or not lines[0].strip():
        raise SIPParseError("missing start line")

    msg = SIPMessage()
    start = lines[0]
    if start.startswith("SIP/"):
        parts = start.split(" ", 2)
        if len(parts) < 2:
            raise SIPParseError(f"malformed status line: {start!r}")
        msg.version = parts[0]
        try:
            msg.status_code = int(parts[1])
        except ValueError as exc:
            raise SIPParseError(f"invalid status code: {parts[1]!r}") from exc
        msg.reason_phrase = parts[2] if len(parts) > 2 else ""
    else:
        parts = start.split(" ")
        if len(parts) != 3:
            raise SIPParseError(f"malformed request line: {start!r}")
        msg.method, msg.request_uri, msg.version = parts
        if strict and msg.method not in KNOWN_METHODS:
            raise SIPParseError(f"unknown method: {msg.method!r}")

    for ln in lines[1:]:
        if not ln:
            continue
        if ":" not in ln:
            if strict:
                raise SIPParseError(f"malformed header line: {ln!r}")
            continue
        name, _, value = ln.partition(":")
        msg.headers.add(name.strip(), value.strip())

    # Honour declared Content-Length when present (truncate/accept body).
    cl = msg.headers.get("Content-Length")
    if cl is not None:
        try:
            n = int(cl)
        except ValueError as exc:
            if strict:
                raise SIPParseError(f"invalid Content-Length: {cl!r}") from exc
            n = len(body)
        if n >= 0:
            if strict and n != len(body):
                raise SIPParseError(
                    f"Content-Length {n} does not match body {len(body)}"
                )
            body = body[:n]
    msg.body = body
    return msg


# ---------------------------------------------------------------------------
# Parameter + identifier helpers
# ---------------------------------------------------------------------------

def _extract_param(header_value: str, param: str) -> str | None:
    """Extract ``;param=value`` from a header value (handles quotes)."""
    for seg in header_value.split(";")[1:]:
        if "=" in seg:
            k, _, v = seg.partition("=")
            if k.strip().lower() == param.lower():
                return v.strip().strip('"')
        elif seg.strip().lower() == param.lower():
```

```
            return ""
    return None


def new_branch() -> str:
    """Generate an RFC 3261 compliant unique Via branch."""
    return MAGIC_COOKIE + secrets.token_hex(12)


def new_call_id(host: str = "siptestwin.local") -> str:
    return f"{secrets.token_hex(12)}@{host}"


def new_tag() -> str:
    return secrets.token_hex(6)


# ---------------------------------------------------------------------------
# Request / response builders
# ---------------------------------------------------------------------------

def build_request(
    method: str,
    request_uri: str,
    *,
    from_uri: str,
    to_uri: str,
    call_id: str,
    cseq: int,
    via_host: str,
    via_port: int,
    transport: str = "UDP",
    from_tag: str | None = None,
    to_tag: str | None = None,
    contact: str | None = None,
    branch: str | None = None,
    max_forwards: int = 70,
    extra_headers: dict[str, str] | None = None,
    body: bytes = b"",
    user_agent: str = "SIPTestWin/0.1",
) -> SIPMessage:
    """Build a well-formed SIP request with all mandatory headers."""
    msg = SIPMessage(method=method.upper(), request_uri=request_uri)
    branch = branch or new_branch()
    msg.headers.add(
        "Via",
        f"SIP/2.0/{transport.upper()} {via_host}:{via_port};branch={branch};rport",
    )
    msg.headers.add("Max-Forwards", str(max_forwards))
    from_val = from_uri
    if from_tag:
        from_val += f";tag={from_tag}"
    msg.headers.add("From", from_val)
    to_val = to_uri
    if to_tag:
        to_val += f";tag={to_tag}"
    msg.headers.add("To", to_val)
    msg.headers.add("Call-ID", call_id)
    msg.headers.add("CSeq", f"{cseq} {method.upper()}")
    if contact:
        msg.headers.add("Contact", contact)
    msg.headers.add("User-Agent", user_agent)
    if extra_headers:
        for k, v in extra_headers.items():
            msg.headers.add(k, v)
    msg.body = body
    msg.headers.set("Content-Length", str(len(body)))
    return msg


def build_response(
    request: SIPMessage,
    status_code: int,
    reason_phrase: str | None = None,
    *,
    to_tag: str | None = None,
    contact: str | None = None,
    extra_headers: dict[str, str] | None = None,
    body: bytes = b"",
) -> SIPMessage:
    """Build a response to ``request`` echoing mandatory headers (RFC 3261 8.2.6)."""
    resp = SIPMessage(status_code=status_code, reason_phrase=reason_phrase or _reason(status_code))
    # Copy Via (all), From, Call-ID, CSeq verbatim; To gets a tag for non-100.
    for via in request.headers.get_all("Via"):
```

```
        resp.headers.add("Via", via)
    resp.headers.add("From", request.headers.get("From", ""))
    to_val = request.headers.get("To", "")
    if to_tag and "tag=" not in to_val and status_code != 100:
        to_val += f";tag={to_tag}"
    resp.headers.add("To", to_val)
    resp.headers.add("Call-ID", request.headers.get("Call-ID", ""))
    resp.headers.add("CSeq", request.headers.get("CSeq", ""))
    if contact:
        resp.headers.add("Contact", contact)
    if extra_headers:
        for k, v in extra_headers.items():
            resp.headers.add(k, v)
    resp.body = body
    resp.headers.set("Content-Length", str(len(body)))
    return resp


_REASONS = {
    100: "Trying", 180: "Ringing", 181: "Call Is Being Forwarded",
    182: "Queued", 183: "Session Progress",
    200: "OK", 202: "Accepted",
    300: "Multiple Choices", 301: "Moved Permanently", 302: "Moved Temporarily",
    305: "Use Proxy", 380: "Alternative Service",
    400: "Bad Request", 401: "Unauthorized", 403: "Forbidden", 404: "Not Found",
    405: "Method Not Allowed", 406: "Not Acceptable", 407: "Proxy Authentication Required",
    408: "Request Timeout", 413: "Request Entity Too Large",
    414: "Request-URI Too Long", 415: "Unsupported Media Type",
    420: "Bad Extension", 423: "Interval Too Brief", 480: "Temporarily Unavailable",
    481: "Call/Transaction Does Not Exist", 482: "Loop Detected", 483: "Too Many Hops",
    486: "Busy Here", 487: "Request Terminated", 488: "Not Acceptable Here",
    500: "Server Internal Error", 501: "Not Implemented", 503: "Service Unavailable",
    504: "Server Time-out", 600: "Busy Everywhere", 603: "Decline",
}


def _reason(code: int) -> str:
    return _REASONS.get(code, "Unknown")


def generate_sdp(host: str, port: int = 8000, session_id: int | None = None) -> bytes:
    """Produce a minimal, valid SDP offer body for INVITE testing."""
    sid = session_id if session_id is not None else int(time.time())
    sdp = (
        "v=0\r\n"
        f"o=siptestwin {sid} {sid} IN IP4 {host}\r\n"
        "s=SIPTestWin Session\r\n"
        f"c=IN IP4 {host}\r\n"
        "t=0 0\r\n"
        f"m=audio {port} RTP/AVP 0 8\r\n"
        "a=rtpmap:0 PCMU/8000\r\n"
        "a=rtpmap:8 PCMA/8000\r\n"
        "a=sendrecv\r\n"
    )
    return sdp.encode("utf-8")


def process_id() -> int:
    """Return the current process id (used for deterministic session ids)."""
    return os.getpid()
```

src/siptestwin/transaction.py

Maquina de estados de la transaccion cliente (RFC 3261 seccion 17): emparejamiento por branch y metodo CSeq, temporizadores T1/T2/B/F y calendario de retransmision en UDP. *(124 lineas)*

```
"""SIP transaction state machine (RFC 3261 section 17).

A SIP *transaction* is the exchange of a single request and its responses,
matched by the Via branch parameter and the CSeq method. This is distinct from
a *dialog* / call session (see :mod:`siptestwin.dialog`): one dialog can span
several transactions (INVITE, then BYE, etc.).

This module implements the client-side transaction timers and state handling
used by the test engine (UAC). Server transaction handling lives in the test
server. Timers follow RFC 3261 defaults and apply retransmissions only on
unreliable (UDP) transports.
"""

from __future__ import annotations
```

```
15 |
16 | import enum
17 | import time
18 | from dataclasses import dataclass, field
19 |
20 | from .sip_message import SIPMessage
21 |
22 | # RFC 3261 timer defaults (seconds).
23 | T1 = 0.5
24 | T2 = 4.0
25 | T4 = 5.0
26 | TIMER_B = 64 * T1   # INVITE client transaction timeout
27 | TIMER_F = 64 * T1   # non-INVITE client transaction timeout
28 | TIMER_D = 32.0      # wait in Completed (INVITE, UDP)
29 | TIMER_K = T4        # wait in Completed (non-INVITE, UDP)
30 |
31 |
32 | class TxState(enum.Enum):
33 |     CALLING = "calling"        # INVITE only
34 |     TRYING = "trying"          # non-INVITE only
35 |     PROCEEDING = "proceeding"
36 |     COMPLETED = "completed"
37 |     TERMINATED = "terminated"
38 |
39 |
40 | @dataclass
41 | class ClientTransaction:
42 |     """Tracks one outgoing request and the matching responses."""
43 |
44 |     request: SIPMessage
45 |     branch: str
46 |     method: str
47 |     reliable: bool
48 |     created_at: float = field(default_factory=time.monotonic)
49 |     state: TxState = field(init=False)
50 |     last_response: SIPMessage | None = None
51 |     responses: list[SIPMessage] = field(default_factory=list)
52 |     retransmit_count: int = 0
53 |     _next_interval: float = T1
54 |     _last_tx_at: float = field(default_factory=time.monotonic)
55 |
56 |     def __post_init__(self) -> None:
57 |         self.state = TxState.CALLING if self.is_invite else TxState.TRYING
58 |
59 |     @property
60 |     def is_invite(self) -> bool:
61 |         return self.method == "INVITE"
62 |
63 |     # ----- matching -----------------------------------------------------
64 |     def matches(self, response: SIPMessage) -> bool:
65 |         """RFC 3261 section 17.1.3 transaction matching (client side)."""
66 |         if not response.is_response:
67 |             return False
68 |         if response.top_via_branch() != self.branch:
69 |             return False
70 |         rmethod = response.cseq_method
71 |         # CANCEL and INVITE share a branch but are different transactions.
72 |         return rmethod == self.method
73 |
74 |     # ----- events ------------------------------------------------------
75 |     def on_response(self, response: SIPMessage) -> None:
76 |         """Advance the state machine for a received response."""
77 |         self.last_response = response
78 |         self.responses.append(response)
79 |         code = response.status_code or 0
80 |         if self.is_invite:
81 |             if code < 200:
82 |                 self.state = TxState.PROCEEDING
83 |             elif 200 <= code < 300:
84 |                 # 2xx terminates the INVITE client transaction immediately;
85 |                 # ACK for 2xx is a separate transaction handled by the dialog.
86 |                 self.state = TxState.TERMINATED
87 |             else:
88 |                 self.state = TxState.COMPLETED  # must ACK the failure response
89 |         else:
90 |             if code < 200:
91 |                 self.state = TxState.PROCEEDING
92 |             else:
93 |                 self.state = TxState.COMPLETED
94 |
95 |     # ----- retransmission (UDP only) ------------------------------------
96 |     def due_for_retransmit(self, now: float) -> bool:
97 |         if self.reliable:
98 |             return False
```

```
        if self.state not in (TxState.CALLING, TxState.TRYING, TxState.PROCEEDING):
            return False
        # Non-INVITE stops retransmitting once provisional received.
        if not self.is_invite and self.state == TxState.PROCEEDING:
            return False
        return (now - self._last_tx_at) >= self._next_interval

    def note_retransmit(self, now: float) -> None:
        self.retransmit_count += 1
        self._last_tx_at = now
        if self.is_invite:
            # INVITE: interval doubles each time (T1, 2T1, 4T1, ...).
            self._next_interval = min(self._next_interval * 2, TIMER_B)
        else:
            # non-INVITE: double up to T2, then cap at T2.
            self._next_interval = min(self._next_interval * 2, T2)

    def timed_out(self, now: float) -> bool:
        if self.state in (TxState.COMPLETED, TxState.TERMINATED):
            return False
        limit = TIMER_B if self.is_invite else TIMER_F
        return (now - self.created_at) >= limit

    @property
    def is_done(self) -> bool:
        return self.state in (TxState.COMPLETED, TxState.TERMINATED)
```

src/siptestwin/dialog.py

Estado del dialogo/sesion de llamada (seccion 12): Call-ID + tags, CSeq y destino remoto; construye ACK y BYE dentro del dialogo. *(100 lineas)*

```
"""SIP dialog / call session state (RFC 3261 section 12).

A dialog is a peer-to-peer relationship identified by Call-ID plus the local
and remote tags. It persists across the transactions of a call (INVITE ->
200 -> ACK ... BYE). This holds the state needed to build in-dialog requests
(ACK, BYE) with correct tags, CSeq and target.
"""

from __future__ import annotations

from dataclasses import dataclass, field

from . import sip_message as sip
from .sip_message import SIPMessage


@dataclass
class Dialog:
    call_id: str
    local_uri: str
    remote_uri: str
    local_tag: str
    remote_tag: str | None = None
    local_seq: int = 1
    remote_target: str | None = None   # Contact URI from the peer
    route_set: list[str] = field(default_factory=list)
    secure: bool = False
    confirmed: bool = False

    @classmethod
    def from_invite_response(
        cls,
        request: SIPMessage,
        response: SIPMessage,
        *,
        local_uri: str,
        remote_uri: str,
    ) -> "Dialog":
        """Create a dialog from an INVITE and its dialog-establishing response."""
        d = cls(
            call_id=request.call_id or "",
            local_uri=local_uri,
            remote_uri=remote_uri,
            local_tag=request.from_tag() or sip.new_tag(),
            remote_tag=response.to_tag(),
            local_seq=request.cseq_number or 1,
        )
        contact = response.headers.get("Contact")
```

```
        if contact:
            d.remote_target = _uri_from_contact(contact)
        # Record-Route set (reversed) would populate route_set; omitted for the
        # direct UAC<->UAS lab topology this tool targets.
        d.confirmed = bool(response.status_code and response.status_code >= 200)
        return d

    def _target_uri(self) -> str:
        return self.remote_target or self.remote_uri

    def build_ack(self, invite: SIPMessage, response: SIPMessage,
                  via_host: str, via_port: int, transport: str) -> SIPMessage:
        """Build an ACK for a 2xx response (new transaction, same CSeq num)."""
        ack = sip.build_request(
            "ACK",
            self._target_uri(),
            from_uri=self.local_uri,
            to_uri=self.remote_uri,
            call_id=self.call_id,
            cseq=invite.cseq_number or self.local_seq,
            via_host=via_host,
            via_port=via_port,
            transport=transport,
            from_tag=self.local_tag,
            to_tag=self.remote_tag,
        )
        return ack

    def build_bye(self, via_host: str, via_port: int, transport: str) -> SIPMessage:
        """Build a BYE to tear down a confirmed dialog (new transaction)."""
        self.local_seq += 1
        return sip.build_request(
            "BYE",
            self._target_uri(),
            from_uri=self.local_uri,
            to_uri=self.remote_uri,
            call_id=self.call_id,
            cseq=self.local_seq,
            via_host=via_host,
            via_port=via_port,
            transport=transport,
            from_tag=self.local_tag,
            to_tag=self.remote_tag,
        )


def _uri_from_contact(contact: str) -> str:
    """Extract the SIP URI from a Contact header value (strip display + <>)."""
    value = contact.strip()
    if "<" in value and ">" in value:
        return value[value.index("<") + 1: value.index(">")]
    return value.split(";")[0].strip()
```

src/siptestwin/transport/__init__.py

Paquete de transportes: exporta la interfaz y la factoria. *(11 lineas)*

```
"""Pluggable SIP transports (UDP, TCP; TLS optional).

Transports move bytes; they have no knowledge of SIP semantics. A transport
delivers inbound data to a handler callback ``handler(data: bytes, addr)`` and
exposes ``send(data, addr=None)``.
"""

from .base import Transport, TransportError
from .factory import create_transport

__all__ = ["Transport", "TransportError", "create_transport"]
```

src/siptestwin/transport/base.py

Interfaz Transport y utilidades compartidas (dispatch, cierre). *(45 lineas)*

```
"""Transport base class and shared types."""

from __future__ import annotations

import threading
```

```
from typing import Callable, Protocol

# handler(data, addr) -> None
Handler = Callable[[bytes, tuple[str, int]], None]


class TransportError(RuntimeError):
    """Raised for transport-level failures (bind, connect, send)."""


class Transport(Protocol):
    """Minimal transport interface used by the engine and test server."""

    name: str

    def start(self, handler: Handler) -> None:
        """Begin receiving; deliver inbound data to ``handler``."""

    def send(self, data: bytes, addr: tuple[str, int] | None = None) -> None:
        """Send ``data``. ``addr`` may be ignored by connection-oriented transports."""

    def local_addr(self) -> tuple[str, int]:
        """Return the bound ``(host, port)``."""

    def close(self) -> None:
        """Release all resources. Idempotent."""


class BaseTransport:
    """Common bookkeeping shared by concrete transports."""

    def __init__(self) -> None:
        self._handler: Handler | None = None
        self._closed = threading.Event()
        self._lock = threading.Lock()

    def _dispatch(self, data: bytes, addr: tuple[str, int]) -> None:
        handler = self._handler
        if handler is not None and not self._closed.is_set():
            handler(data, addr)
```

src/siptestwin/transport/udp.py

Transporte UDP (sin conexion), con hilo de recepcion. *(79 lineas)*

```
"""UDP transport (connectionless)."""

from __future__ import annotations

import socket
import threading

from .base import BaseTransport, Handler, TransportError

MAX_DATAGRAM = 65535


class UDPTransport(BaseTransport):
    """A UDP socket that can both send to a peer and receive datagrams.

    Works on Windows without special privileges. Binding to port 0 lets the OS
    pick an ephemeral source port, which is the default for a test client.
    """

    name = "udp"

    def __init__(self, bind_host: str = "0.0.0.0", bind_port: int = 0) -> None:
        super().__init__()
        self._bind = (bind_host, bind_port)
        self._sock: socket.socket | None = None
        self._thread: threading.Thread | None = None

    def start(self, handler: Handler) -> None:
        self._handler = handler
        try:
            sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            sock.bind(self._bind)
        except OSError as exc:
            raise TransportError(f"UDP bind failed on {self._bind}: {exc}") from exc
        sock.settimeout(0.3)
```

```
        self._sock = sock
        self._thread = threading.Thread(target=self._recv_loop, name="udp-recv", daemon=True)
        self._thread.start()

    def _recv_loop(self) -> None:
        assert self._sock is not None
        while not self._closed.is_set():
            try:
                data, addr = self._sock.recvfrom(MAX_DATAGRAM)
            except socket.timeout:
                continue
            except OSError:
                break
            if data:
                self._dispatch(data, addr)

    def send(self, data: bytes, addr: tuple[str, int] | None = None) -> None:
        if self._sock is None:
            raise TransportError("UDP transport not started")
        if addr is None:
            raise TransportError("UDP send requires a destination address")
        if len(data) > MAX_DATAGRAM:
            raise TransportError(f"datagram too large: {len(data)} bytes")
        try:
            self._sock.sendto(data, addr)
        except OSError as exc:
            raise TransportError(f"UDP send to {addr} failed: {exc}") from exc

    def local_addr(self) -> tuple[str, int]:
        if self._sock is None:
            return self._bind
        return self._sock.getsockname()

    def close(self) -> None:
        self._closed.set()
        if self._thread and self._thread.is_alive():
            self._thread.join(timeout=1.0)
        if self._sock is not None:
            with self._lock:
                try:
                    self._sock.close()
                finally:
                    self._sock = None
```

src/siptestwin/transport/tcp.py

Transporte TCP con enmarcado de mensajes por Content-Length; modos cliente (connect) y servidor (listen); TLS opcional. *(220 lineas)*

```
"""TCP transport with SIP message framing (client connect and server listen).

SIP over a stream transport requires framing each message by its headers plus
``Content-Length`` (RFC 3261 section 18). This transport reassembles the
stream and dispatches one complete SIP message at a time.
"""

from __future__ import annotations

import socket
import ssl
import threading

from .base import BaseTransport, Handler, TransportError

_RECV = 65536


def _frame_messages(buffer: bytearray) -> list[bytes]:
    """Pop all complete SIP messages from ``buffer`` (mutated in place)."""
    messages: list[bytes] = []
    while True:
        sep = buffer.find(b"\r\n\r\n")
        sep_len = 4
        if sep == -1:
            sep = buffer.find(b"\n\n")
            sep_len = 2
            if sep == -1:
                break
        head = bytes(buffer[:sep])
        content_length = 0
```

```
        for line in head.split(b"\n"):
            low = line.strip().lower()
            if low.startswith(b"content-length:") or low.startswith(b"l:"):
                try:
                    content_length = int(line.split(b":", 1)[1].strip())
                except (ValueError, IndexError):
                    content_length = 0
                break
        total = sep + sep_len + content_length
        if len(buffer) < total:
            break
        messages.append(bytes(buffer[:total]))
        del buffer[:total]
    return messages


class TCPTransport(BaseTransport):
    """TCP transport. In client mode it connects to a single peer; in server
    mode it listens and services inbound connections."""

    name = "tcp"

    def __init__(
        self,
        *,
        peer: tuple[str, int] | None = None,
        bind: tuple[str, int] | None = None,
        listen: bool = False,
        tls: bool = False,
        verify_tls: bool = True,
        server_side_cert: tuple[str, str] | None = None,
    ) -> None:
        super().__init__()
        self._peer = peer
        self._bind = bind or ("0.0.0.0", 0)
        self._listen = listen
        self._tls = tls
        self._verify_tls = verify_tls
        self._server_cert = server_side_cert
        self.name = "tls" if tls else "tcp"
        self._sock: socket.socket | None = None
        self._conns: dict[tuple[str, int], socket.socket] = {}
        self._threads: list[threading.Thread] = []
        self._bound_addr = self._bind

    # ----- startup -------------------------------------------------------
    def start(self, handler: Handler) -> None:
        self._handler = handler
        if self._listen:
            self._start_server()
        else:
            self._start_client()

    def _wrap_client(self, sock: socket.socket, host: str) -> socket.socket:
        if not self._tls:
            return sock
        ctx = ssl.create_default_context()
        if not self._verify_tls:
            ctx.check_hostname = False
            ctx.verify_mode = ssl.CERT_NONE
        return ctx.wrap_socket(sock, server_hostname=host)

    def _start_client(self) -> None:
        if self._peer is None:
            raise TransportError("TCP client requires a peer address")
        host, port = self._peer
        try:
            sock = socket.create_connection((host, port), timeout=10)
            sock = self._wrap_client(sock, host)
        except OSError as exc:
            raise TransportError(f"TCP connect to {self._peer} failed: {exc}") from exc
        sock.settimeout(0.3)
        self._sock = sock
        self._bound_addr = sock.getsockname()[:2]
        self._conns[self._peer] = sock
        t = threading.Thread(target=self._conn_loop, args=(sock, self._peer),
                             name="tcp-client", daemon=True)
        t.start()
        self._threads.append(t)

    def _start_server(self) -> None:
        try:
            srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

```
            srv.bind(self._bind)
            srv.listen(16)
        except OSError as exc:
            raise TransportError(f"TCP listen on {self._bind} failed: {exc}") from exc
        srv.settimeout(0.3)
        if self._tls:
            if not self._server_cert:
                raise TransportError("TLS server requires (certfile, keyfile)")
            ctx = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
            ctx.load_cert_chain(certfile=self._server_cert[0], keyfile=self._server_cert[1])
            self._ssl_ctx: ssl.SSLContext | None = ctx
        else:
            self._ssl_ctx = None
        self._sock = srv
        self._bound_addr = srv.getsockname()[:2]
        t = threading.Thread(target=self._accept_loop, name="tcp-accept", daemon=True)
        t.start()
        self._threads.append(t)

    def _accept_loop(self) -> None:
        assert self._sock is not None
        while not self._closed.is_set():
            try:
                conn, addr = self._sock.accept()
            except socket.timeout:
                continue
            except OSError:
                break
            try:
                if self._ssl_ctx is not None:
                    conn = self._ssl_ctx.wrap_socket(conn, server_side=True)
            except ssl.SSLError:
                conn.close()
                continue
            conn.settimeout(0.3)
            with self._lock:
                self._conns[addr[:2]] = conn
            ct = threading.Thread(target=self._conn_loop, args=(conn, addr[:2]),
                                  name="tcp-conn", daemon=True)
            ct.start()
            self._threads.append(ct)

    def _conn_loop(self, conn: socket.socket, peer: tuple[str, int]) -> None:
        buffer = bytearray()
        while not self._closed.is_set():
            try:
                chunk = conn.recv(_RECV)
            except socket.timeout:
                continue
            except OSError:
                break
            if not chunk:
                break
            buffer.extend(chunk)
            if len(buffer) > 1_048_576:  # 1 MiB stream safety cap
                buffer.clear()
                break
            for msg in _frame_messages(buffer):
                self._dispatch(msg, peer)
        with self._lock:
            self._conns.pop(peer, None)
        try:
            conn.close()
        except OSError:
            pass

    # ----- send ------------------------------------------------------------
    def send(self, data: bytes, addr: tuple[str, int] | None = None) -> None:
        conn: socket.socket | None
        if addr is not None:
            conn = self._conns.get(addr) or self._conns.get(self._peer) if self._peer else None
            conn = self._conns.get(addr, conn)
        elif self._peer is not None:
            conn = self._conns.get(self._peer)
        else:
            conn = next(iter(self._conns.values()), None)
        if conn is None:
            raise TransportError(f"no TCP connection to {addr or self._peer}")
        try:
            conn.sendall(data)
        except OSError as exc:
            raise TransportError(f"TCP send failed: {exc}") from exc

    def local_addr(self) -> tuple[str, int]:
```

```
        return self._bound_addr

    def close(self) -> None:
        self._closed.set()
        with self._lock:
            conns = list(self._conns.values())
            self._conns.clear()
        for c in conns:
            try:
                c.close()
            except OSError:
                pass
        if self._sock is not None:
            try:
                self._sock.close()
            except OSError:
                pass
            self._sock = None
        for t in self._threads:
            if t.is_alive():
                t.join(timeout=1.0)
```

src/siptestwin/transport/factory.py

Factoria que crea el transporte cliente segun la configuracion. *(30 lineas)*

```
"""Factory for building transports from configuration."""

from __future__ import annotations

from .base import Transport, TransportError
from .tcp import TCPTransport
from .udp import UDPTransport


def create_transport(
    transport: str,
    *,
    peer: tuple[str, int] | None = None,
    bind_host: str = "0.0.0.0",
    bind_port: int = 0,
    verify_tls: bool = True,
) -> Transport:
    """Create a client-mode transport of the requested kind.

    ``transport`` is one of ``udp``, ``tcp`` or ``tls`` (case-insensitive).
    """
    kind = transport.lower()
    if kind == "udp":
        return UDPTransport(bind_host=bind_host, bind_port=bind_port)
    if kind == "tcp":
        return TCPTransport(peer=peer, bind=(bind_host, bind_port))
    if kind == "tls":
        return TCPTransport(peer=peer, bind=(bind_host, bind_port), tls=True,
                            verify_tls=verify_tls)
    raise TransportError(f"unsupported transport: {transport!r}")
```

src/siptestwin/authentication/__init__.py

Paquete de autenticacion: exporta Digest. *(5 lineas)*

```
"""SIP authentication helpers (Digest, RFC 2617 / RFC 3261 section 22)."""

from .digest import DigestChallenge, build_authorization, parse_challenge

__all__ = ["DigestChallenge", "build_authorization", "parse_challenge"]
```

src/siptestwin/authentication/digest.py

Autenticacion Digest (RFC 2617 / 3261 seccion 22): parsea el desafio y calcula la respuesta con credenciales de laboratorio. Sin fuerza bruta. *(153 lineas)*

```
"""SIP Digest authentication (RFC 2617, RFC 3261 section 22).

```

```
This computes the response to a server's Digest challenge using credentials
supplied for an authorized lab account. It intentionally supports only
answering a single, legitimate challenge with known credentials. It contains
no credential-guessing or brute-force functionality.

Supported algorithms: MD5, MD5-sess. Supported qop: absent and "auth".
"""

from __future__ import annotations

import hashlib
import secrets
from dataclasses import dataclass


class DigestError(ValueError):
    """Raised for unsupported or malformed Digest parameters."""


def _parse_params(value: str) -> dict[str, str]:
    """Parse a comma-separated list of ``key=value`` auth parameters."""
    params: dict[str, str] = {}
    i = 0
    n = len(value)
    while i < n:
        # skip whitespace / commas
        while i < n and value[i] in ", \t":
            i += 1
        start = i
        while i < n and value[i] != "=":
            i += 1
        key = value[start:i].strip().lower()
        if i >= n:
            break
        i += 1  # skip '='
        if i < n and value[i] == '"':
            i += 1
            vstart = i
            while i < n and value[i] != '"':
                i += 1
            val = value[vstart:i]
            i += 1  # skip closing quote
        else:
            vstart = i
            while i < n and value[i] != ",":
                i += 1
            val = value[vstart:i].strip()
        if key:
            params[key] = val
    return params


@dataclass
class DigestChallenge:
    """A parsed WWW-Authenticate / Proxy-Authenticate Digest challenge."""

    scheme: str
    realm: str
    nonce: str
    algorithm: str = "MD5"
    qop: str | None = None
    opaque: str | None = None
    stale: bool = False
    is_proxy: bool = False


def parse_challenge(header_value: str, *, is_proxy: bool = False) -> DigestChallenge:
    """Parse a ``WWW-Authenticate``/``Proxy-Authenticate`` header value."""
    value = header_value.strip()
    scheme, _, rest = value.partition(" ")
    if scheme.lower() != "digest":
        raise DigestError(f"unsupported auth scheme: {scheme!r}")
    params = _parse_params(rest)
    if "realm" not in params or "nonce" not in params:
        raise DigestError("Digest challenge missing realm or nonce")
    return DigestChallenge(
        scheme="Digest",
        realm=params["realm"],
        nonce=params["nonce"],
        algorithm=params.get("algorithm", "MD5"),
        qop=params.get("qop"),
        opaque=params.get("opaque"),
        stale=params.get("stale", "false").lower() == "true",
        is_proxy=is_proxy,
```

```
    )

def _h(data: str, algorithm: str) -> str:
    algo = algorithm.lower().replace("-sess", "")
    if algo == "md5":
        return hashlib.md5(data.encode("utf-8")).hexdigest()
    if algo == "sha-256":
        return hashlib.sha256(data.encode("utf-8")).hexdigest()
    raise DigestError(f"unsupported Digest algorithm: {algorithm!r}")


def build_authorization(
    challenge: DigestChallenge,
    *,
    username: str,
    password: str,
    method: str,
    uri: str,
    cnonce: str | None = None,
    nc: int = 1,
) -> str:
    """Compute the Authorization / Proxy-Authorization header value.

    Returns the full header value string (starting with ``Digest ``). The
    caller adds it under the correct header name.
    """
    algorithm = challenge.algorithm
    qop = None
    if challenge.qop:
        offered = [q.strip().lower() for q in challenge.qop.split(",")]
        if "auth" in offered:
            qop = "auth"
        else:
            raise DigestError(f"unsupported qop offered: {challenge.qop!r}")

    ha1 = _h(f"{username}:{challenge.realm}:{password}", algorithm)
    if algorithm.lower().endswith("-sess"):
        cnonce = cnonce or secrets.token_hex(8)
        ha1 = _h(f"{ha1}:{challenge.nonce}:{cnonce}", algorithm)

    ha2 = _h(f"{method}:{uri}", algorithm)

    if qop == "auth":
        cnonce = cnonce or secrets.token_hex(8)
        nc_value = f"{nc:08x}"
        response = _h(
            f"{ha1}:{challenge.nonce}:{nc_value}:{cnonce}:{qop}:{ha2}", algorithm
        )
    else:
        response = _h(f"{ha1}:{challenge.nonce}:{ha2}", algorithm)

    parts = [
        f'username="{username}"',
        f'realm="{challenge.realm}"',
        f'nonce="{challenge.nonce}"',
        f'uri="{uri}"',
        f'response="{response}"',
        f"algorithm={algorithm}",
    ]
    if qop == "auth":
        parts.append("qop=auth")
        parts.append(f"nc={nc:08x}")
        parts.append(f'cnonce="{cnonce}"')
    if challenge.opaque is not None:
        parts.append(f'opaque="{challenge.opaque}"')
    return "Digest " + ", ".join(parts)
```

src/siptestwin/config.py

Modelo de configuracion (dataclasses), carga YAML/JSON y validacion estricta (tipos, rangos, topes operativos). Incluye las secciones message (parametrizar headers/INVITE) y capture (traza). *(293 lineas)*

```
"""Configuration model, loading and validation (YAML / JSON).

All operational parameters live in configuration so the tool never needs code
changes to run a different test. Validation is strict: types, ranges and
mutually incompatible options are checked, and the target must be explicit.
"""

```

```
from __future__ import annotations

import json
import os
from dataclasses import dataclass, field, fields
from pathlib import Path
from typing import Any

import yaml

VALID_TRANSPORTS = {"udp", "tcp", "tls"}
VALID_AUTH = {"disabled", "digest"}
VALID_FORMATS = {"json", "csv"}

# Operational hard limits. These are ceilings the tool refuses to exceed even
# if a config asks for more, to keep tests controlled.
MAX_CALLS = 100_000
MAX_CONCURRENCY = 500
MAX_CALL_RATE = 500.0
MAX_DURATION = 86_400


class ConfigError(ValueError):
    """Raised when configuration is missing, malformed or out of range."""


@dataclass
class TargetConfig:
    host: str
    port: int = 5060
    transport: str = "udp"

    def validate(self, errors: list[str]) -> None:
        if not self.host or not isinstance(self.host, str):
            errors.append("target.host is required and must be a string")
        if not (0 < int(self.port) <= 65535):
            errors.append(f"target.port must be 1-65535, got {self.port}")
        if self.transport.lower() not in VALID_TRANSPORTS:
            errors.append(
                f"target.transport must be one of {sorted(VALID_TRANSPORTS)}, "
                f"got {self.transport!r}"
            )


@dataclass
class IdentityConfig:
    from_user: str = "tester"
    from_domain: str = "lab.example"
    to_user: str = "1001"
    to_domain: str | None = None     # defaults to target host
    contact_user: str = "tester"
    display_name: str | None = None

    def validate(self, errors: list[str]) -> None:
        for name in ("from_user", "to_user", "contact_user"):
            val = getattr(self, name)
            if not val or not isinstance(val, str):
                errors.append(f"identity.{name} must be a non-empty string")


@dataclass
class ExecutionConfig:
    scenario: str = ""
    calls: int = 1
    concurrency: int = 1
    call_rate: float = 1.0
    timeout_seconds: float = 5.0
    duration_seconds: float = 0.0   # 0 == bounded by calls, not time
    local_host: str | None = None   # Via/Contact host; autodetected if None
    local_port: int = 0             # 0 == OS-assigned ephemeral source port

    def validate(self, errors: list[str]) -> None:
        if not self.scenario:
            errors.append("execution.scenario is required")
        if not (1 <= int(self.calls) <= MAX_CALLS):
            errors.append(f"execution.calls must be 1-{MAX_CALLS}, got {self.calls}")
        if not (1 <= int(self.concurrency) <= MAX_CONCURRENCY):
            errors.append(
                f"execution.concurrency must be 1-{MAX_CONCURRENCY}, got {self.concurrency}"
            )
        if not (0 < float(self.call_rate) <= MAX_CALL_RATE):
            errors.append(
                f"execution.call_rate must be >0 and <= {MAX_CALL_RATE}, got {self.call_rate}"
            )
```

```
        if self.timeout_seconds <= 0:
            errors.append("execution.timeout_seconds must be > 0")
        if self.duration_seconds < 0 or self.duration_seconds > MAX_DURATION:
            errors.append(f"execution.duration_seconds must be 0-{MAX_DURATION}")
        if self.concurrency > self.calls:
            errors.append("execution.concurrency cannot exceed execution.calls")


@dataclass
class SecurityConfig:
    authentication: str = "disabled"
    username: str | None = None
    password: str | None = None
    verify_tls: bool = True
    authorized_targets: list[str] = field(default_factory=list)
    require_confirmation: bool = True

    def validate(self, errors: list[str]) -> None:
        if self.authentication.lower() not in VALID_AUTH:
            errors.append(
                f"security.authentication must be one of {sorted(VALID_AUTH)}, "
                f"got {self.authentication!r}"
            )
        if self.authentication.lower() == "digest" and not self.username:
            errors.append("security.authentication=digest requires security.username")


@dataclass
class ReportingConfig:
    output_dir: str = "results"
    formats: list[str] = field(default_factory=lambda: ["json"])

    def validate(self, errors: list[str]) -> None:
        for fmt in self.formats:
            if fmt.lower() not in VALID_FORMATS:
                errors.append(
                    f"reporting.formats entries must be in {sorted(VALID_FORMATS)}, "
                    f"got {fmt!r}"
                )


@dataclass
class InviteConfig:
    """INVITE-specific message customization."""

    content_type: str = "application/sdp"
    sdp: str | None = None            # custom SDP body (overrides the generated one)
    extra_headers: dict[str, str] = field(default_factory=dict)

    def validate(self, errors: list[str]) -> None:
        if not isinstance(self.extra_headers, dict):
            errors.append("message.invite.extra_headers must be a mapping")
        if self.sdp is not None and not isinstance(self.sdp, str):
            errors.append("message.invite.sdp must be a string")


@dataclass
class MessageConfig:
    """Global header / body parametrization applied to built-in requests.

    ``extra_headers`` are added to every built-in request; ``remove_headers``
    are stripped from them; ``invite`` customizes the INVITE body and headers.
    Header values support scenario placeholders (e.g. ``[call_id]``, ``[$var]``).
    """

    extra_headers: dict[str, str] = field(default_factory=dict)
    remove_headers: list[str] = field(default_factory=list)
    invite: InviteConfig = field(default_factory=InviteConfig)

    def validate(self, errors: list[str]) -> None:
        if not isinstance(self.extra_headers, dict):
            errors.append("message.extra_headers must be a mapping")
        if not isinstance(self.remove_headers, list):
            errors.append("message.remove_headers must be a list")
        self.invite.validate(errors)


@dataclass
class CaptureConfig:
    """SIP message tracing / capture options."""

    enabled: bool = False
    console: bool = False          # one line per message to the console
    console_full: bool = False     # also print the full message to the console
```

```
    text: bool = True              # write a human-readable .trace.log
    pcap: bool = False             # write a Wireshark-openable .pcap
    file: str | None = None        # base path; default <output_dir>/trace-<stamp>

    def validate(self, errors: list[str]) -> None:
        for name in ("enabled", "console", "console_full", "text", "pcap"):
            if not isinstance(getattr(self, name), bool):
                errors.append(f"capture.{name} must be a boolean")
        if self.enabled and not (self.console or self.text or self.pcap):
            errors.append("capture.enabled is true but no sink (console/text/pcap) is on")


@dataclass
class Config:
    target: TargetConfig
    identity: IdentityConfig = field(default_factory=IdentityConfig)
    execution: ExecutionConfig = field(default_factory=ExecutionConfig)
    security: SecurityConfig = field(default_factory=SecurityConfig)
    reporting: ReportingConfig = field(default_factory=ReportingConfig)
    message: MessageConfig = field(default_factory=MessageConfig)
    capture: CaptureConfig = field(default_factory=CaptureConfig)
    source_path: str | None = None

    def validate(self) -> list[str]:
        errors: list[str] = []
        self.target.validate(errors)
        self.identity.validate(errors)
        self.execution.validate(errors)
        self.security.validate(errors)
        self.reporting.validate(errors)
        self.message.validate(errors)
        self.capture.validate(errors)
        # TLS verification only meaningful with TLS transport.
        if self.target.transport.lower() != "tls" and not self.security.verify_tls:
            # not an error, but normalize silently
            pass
        return errors

    def validate_or_raise(self) -> "Config":
        errors = self.validate()
        if errors:
            raise ConfigError("Invalid configuration:\n  - " + "\n  - ".join(errors))
        return self


def _section(data: dict[str, Any], key: str) -> dict[str, Any]:
    value = data.get(key, {})
    if value is None:
        return {}
    if not isinstance(value, dict):
        raise ConfigError(f"config section {key!r} must be a mapping")
    return value


def _only_known(cls: type, data: dict[str, Any]) -> dict[str, Any]:
    known = {f.name for f in fields(cls)}
    unknown = set(data) - known
    if unknown:
        raise ConfigError(f"unknown keys in {cls.__name__}: {sorted(unknown)}")
    return data


def from_dict(data: dict[str, Any], *, source_path: str | None = None) -> Config:
    if not isinstance(data, dict):
        raise ConfigError("top-level configuration must be a mapping")
    if "target" not in data:
        raise ConfigError("configuration must define a 'target' section")

    target = TargetConfig(**_only_known(TargetConfig, _section(data, "target")))
    identity = IdentityConfig(**_only_known(IdentityConfig, _section(data, "identity")))
    execution = ExecutionConfig(**_only_known(ExecutionConfig, _section(data, "execution")))
    security = SecurityConfig(**_only_known(SecurityConfig, _section(data, "security")))
    reporting = ReportingConfig(**_only_known(ReportingConfig, _section(data, "reporting")))
    message = _message(_section(data, "message"))
    capture = CaptureConfig(**_only_known(CaptureConfig, _section(data, "capture")))
    return Config(
        target=target,
        identity=identity,
        execution=execution,
        security=security,
        reporting=reporting,
        message=message,
        capture=capture,
        source_path=source_path,
```

```
    )


def _message(data: dict[str, Any]) -> MessageConfig:
    invite_data = data.get("invite", {}) or {}
    if not isinstance(invite_data, dict):
        raise ConfigError("message.invite must be a mapping")
    invite = InviteConfig(**_only_known(InviteConfig, invite_data))
    top = {k: v for k, v in data.items() if k != "invite"}
    _only_known(MessageConfig, top)  # reject unknown top-level message keys
    return MessageConfig(invite=invite, **top)


def load(path: str | os.PathLike[str]) -> Config:
    """Load and parse a YAML or JSON config file (no validation)."""
    p = Path(path)
    if not p.is_file():
        raise ConfigError(f"config file not found: {p}")
    raw = p.read_text(encoding="utf-8")
    if p.suffix.lower() in (".yaml", ".yml"):
        data = yaml.safe_load(raw)
    elif p.suffix.lower() == ".json":
        data = json.loads(raw)
    else:
        # Try YAML first (superset of JSON), fall back to JSON.
        try:
            data = yaml.safe_load(raw)
        except yaml.YAMLError:
            data = json.loads(raw)
    return from_dict(data or {}, source_path=str(p))


def load_and_validate(path: str | os.PathLike[str]) -> Config:
    return load(path).validate_or_raise()
```

src/siptestwin/safety.py

Controles de seguridad: lista blanca de destinos, limites de ejecucion, confirmacion, redaccion de credenciales y registro de auditoria. *(173 lineas)*

```
"""Safety controls for SIPTestWin.

These controls exist to keep the tool usable only against systems the operator
owns or is explicitly authorized to test, and to keep every test bounded:

* an explicit allow-list of authorized targets (no implicit "private IP ==
  allowed" assumption);
* rate / concurrency / duration ceilings enforced on top of config limits;
* a validation-only (dry-run) mode that transmits no packets;
* explicit operator confirmation before any active (packet-sending) test;
* credential redaction so passwords never reach logs or reports;
* an append-only audit log of every active run.

The tool deliberately has no functionality for credential brute-forcing,
unbounded flooding, toll fraud, or automated exploitation of third parties.
"""

from __future__ import annotations

import ipaddress
import json
import re
import socket
import time
from dataclasses import dataclass
from pathlib import Path

from .config import Config

# Values that must never appear verbatim in logs/reports.
_SECRET_KEYS = ("password", "passwd", "secret", "response=", "authorization")
_AUTH_LINE = re.compile(r"(?im)^(Proxy-)?Authorization:.*$")
_WWW_RESP = re.compile(r'(response=")[^"]*(")')


class AuthorizationError(RuntimeError):
    """Raised when a target is not on the authorized allow-list."""


class ConfirmationRequired(RuntimeError):
```

```
    """Raised when an active test is attempted without confirmation."""


def redact(text: str) -> str:
    """Remove credential material from a text blob (for logs/reports)."""
    text = _AUTH_LINE.sub(lambda m: m.group(0).split(":", 1)[0] + ": <redacted>", text)
    text = _WWW_RESP.sub(r'\1<redacted>\2', text)
    return text


def redact_config(config: Config) -> dict:
    """Return a dict view of ``config`` safe to log (password removed)."""
    return {
        "target": vars(config.target),
        "identity": vars(config.identity),
        "execution": vars(config.execution),
        "security": {
            **{k: v for k, v in vars(config.security).items() if k != "password"},
            "password": "<redacted>" if config.security.password else None,
        },
        "reporting": vars(config.reporting),
    }


def _normalize_host(host: str) -> set[str]:
    """Return candidate string forms for ``host`` (name and resolved IPs)."""
    forms = {host.lower()}
    try:
        forms.add(str(ipaddress.ip_address(host)))
    except ValueError:
        try:
            for info in socket.getaddrinfo(host, None):
                forms.add(info[4][0])
        except (socket.gaierror, OSError):
            pass
    return forms


def is_authorized(host: str, authorized_targets: list[str]) -> bool:
    """Check whether ``host`` matches the allow-list.

    Each allow-list entry is a hostname, an IP, or a CIDR network. Matching is
    exact for names and membership-based for networks. There is no implicit
    authorization: an empty allow-list authorizes nothing.
    """
    if not authorized_targets:
        return False
    candidates = _normalize_host(host)
    for entry in authorized_targets:
        entry = entry.strip()
        if not entry:
            continue
        if entry.lower() in candidates:
            return True
        try:
            net = ipaddress.ip_network(entry, strict=False)
        except ValueError:
            continue
        for cand in candidates:
            try:
                if ipaddress.ip_address(cand) in net:
                    return True
            except ValueError:
                continue
    return False


def ensure_authorized(config: Config) -> None:
    """Raise :class:`AuthorizationError` if the target is not allow-listed."""
    host = config.target.host
    if not is_authorized(host, config.security.authorized_targets):
        raise AuthorizationError(
            f"target {host!r} is not in security.authorized_targets. "
            "Add it explicitly to confirm you own or are authorized to test it. "
            "A private IP address is NOT treated as authorization."
        )


@dataclass
class RunLimits:
    """Effective ceilings applied to a run (min of config and hard limits)."""

    calls: int
    concurrency: int
```

```
    call_rate: float
    duration_seconds: float

    @classmethod
    def from_config(cls, config: Config) -> "RunLimits":
        from .config import MAX_CALLS, MAX_CONCURRENCY, MAX_CALL_RATE, MAX_DURATION
        ex = config.execution
        return cls(
            calls=min(int(ex.calls), MAX_CALLS),
            concurrency=min(int(ex.concurrency), MAX_CONCURRENCY),
            call_rate=min(float(ex.call_rate), MAX_CALL_RATE),
            duration_seconds=min(float(ex.duration_seconds), MAX_DURATION),
        )


def confirmation_prompt(config: Config, limits: RunLimits) -> str:
    """Human-readable summary shown before an active test begins."""
    t = config.target
    return (
        "ACTIVE SIP TEST — packets WILL be transmitted\n"
        f"  target     : {t.host}:{t.port} ({t.transport.upper()})\n"
        f"  scenario   : {config.execution.scenario}\n"
        f"  calls      : {limits.calls}\n"
        f"  concurrency: {limits.concurrency}\n"
        f"  call_rate  : {limits.call_rate}/s\n"
        f"  duration   : {limits.duration_seconds or 'bounded by calls'}\n"
        f"  auth       : {config.security.authentication}\n"
    )


class AuditLog:
    """Append-only JSON-lines audit log of active runs."""

    def __init__(self, output_dir: str | Path) -> None:
        self.path = Path(output_dir) / "audit.log"

    def record(self, event: str, config: Config, extra: dict | None = None) -> None:
        self.path.parent.mkdir(parents=True, exist_ok=True)
        entry = {
            "ts": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
            "event": event,
            "target": f"{config.target.host}:{config.target.port}/{config.target.transport}",
            "scenario": config.execution.scenario,
        }
        if extra:
            entry.update(extra)
        line = json.dumps(entry, ensure_ascii=False)
        with self.path.open("a", encoding="utf-8") as fh:
            fh.write(redact(line) + "\n")
```

src/siptestwin/scenario_parser.py

Parser del subconjunto XML compatible con SIPp (send/recv/pause/ereg) y la forma integrada con /. Parseo endurecido (sin DOCTYPE). *(272 lineas)*

```
"""Parser for a documented SIPp-compatible subset of scenario XML.

Supported elements: ``<scenario>``, ``<send>``, ``<recv>``, ``<pause>`` and,
inside ``<recv>``, ``<action><ereg .../></action>`` for safe field extraction.

Two ``<send>`` forms are supported:

* **Built-in** — ``<send request="OPTIONS"/>``: the engine constructs a
  well-formed request from the identity/target configuration.
* **Template** — ``<send>`` with a CDATA SIP message using ``[placeholders]``
  (SIPp style) for full control.

See ``docs/scenarios.md`` for the compatibility matrix. Parsing is hardened:
DOCTYPE / external entities are rejected, and regular expressions are length
limited to avoid pathological patterns.
"""

from __future__ import annotations

import re
from dataclasses import dataclass, field
from pathlib import Path
from xml.etree import ElementTree as ET

MAX_SCENARIO_BYTES = 1_048_576  # 1 MiB
```

```
MAX_REGEX_LEN = 512


class ScenarioError(ValueError):
    """Raised for invalid or unsafe scenario documents."""


@dataclass
class Extract:
    """An ``<ereg>`` field-extraction action."""

    regexp: str
    assign_to: str
    search_in: str = "msg"   # "msg" | "hdr"
    header: str | None = None
    occurrence: int = 1

    def compile(self) -> re.Pattern[str]:
        if len(self.regexp) > MAX_REGEX_LEN:
            raise ScenarioError(f"regexp too long (>{MAX_REGEX_LEN} chars)")
        try:
            return re.compile(self.regexp)
        except re.error as exc:
            raise ScenarioError(f"invalid regexp {self.regexp!r}: {exc}") from exc


@dataclass
class Send:
    builtin_method: str | None = None
    template: str | None = None
    retrans_ms: int | None = None
    label: str | None = None
    # Per-send header/body parametrization (built-in form only):
    set_headers: dict[str, str] = field(default_factory=dict)
    remove_headers: list[str] = field(default_factory=list)
    body: str | None = None
    content_type: str | None = None

    @property
    def kind(self) -> str:
        return "send"


@dataclass
class Recv:
    response: int | None = None
    request: str | None = None
    optional: bool = False
    timeout_ms: int | None = None
    actions: list[Extract] = field(default_factory=list)
    label: str | None = None

    @property
    def kind(self) -> str:
        return "recv"


@dataclass
class Pause:
    milliseconds: int
    label: str | None = None

    @property
    def kind(self) -> str:
        return "pause"


Step = Send | Recv | Pause


@dataclass
class Scenario:
    name: str
    steps: list[Step]

    def summary(self) -> str:
        parts = []
        for s in self.steps:
            if isinstance(s, Send):
                parts.append(f"send({s.builtin_method or 'template'})")
            elif isinstance(s, Recv):
                tgt = s.response if s.response is not None else s.request
                parts.append(f"recv({tgt}{'?' if s.optional else ''})")
            else:
```

```
                parts.append(f"pause({s.milliseconds}ms)")
        return " -> ".join(parts)


def _guard_xml(text: str) -> None:
    lowered = text.lower()
    if "<!doctype" in lowered or "<!entity" in lowered:
        raise ScenarioError("DOCTYPE / entity declarations are not allowed in scenarios")


def _bool_attr(el: ET.Element, name: str, default: bool = False) -> bool:
    val = el.get(name)
    if val is None:
        return default
    return val.strip().lower() in ("true", "1", "yes")


def _int_attr(el: ET.Element, name: str) -> int | None:
    val = el.get(name)
    if val is None:
        return None
    try:
        return int(val)
    except ValueError as exc:
        raise ScenarioError(f"attribute {name!r} must be an integer, got {val!r}") from exc


def parse_string(text: str, *, name: str = "<string>") -> Scenario:
    if len(text.encode("utf-8")) > MAX_SCENARIO_BYTES:
        raise ScenarioError("scenario document exceeds size limit")
    _guard_xml(text)
    try:
        root = ET.fromstring(text)
    except ET.ParseError as exc:
        raise ScenarioError(f"XML parse error: {exc}") from exc
    if root.tag != "scenario":
        raise ScenarioError(f"root element must be <scenario>, got <{root.tag}>")

    scenario_name = root.get("name", name)
    steps: list[Step] = []
    for el in list(root):
        if el.tag == "send":
            steps.append(_parse_send(el))
        elif el.tag == "recv":
            steps.append(_parse_recv(el))
        elif el.tag == "pause":
            ms = _int_attr(el, "milliseconds")
            if ms is None:
                ms = _int_attr(el, "ms") or 0
            if ms < 0:
                raise ScenarioError("pause milliseconds must be >= 0")
            steps.append(Pause(milliseconds=ms, label=el.get("label")))
        elif el.tag in ("Reference", "nop"):
            continue  # tolerated no-ops
        elif el.tag in ("sendCmd", "recvCmd"):
            raise ScenarioError(
                f"<{el.tag}> (external command control) is not supported for safety"
            )
        else:
            raise ScenarioError(f"unsupported scenario element <{el.tag}>")

    if not steps:
        raise ScenarioError("scenario contains no steps")
    _validate_flow(steps)
    return Scenario(name=scenario_name, steps=steps)


def _parse_send(el: ET.Element) -> Send:
    method = el.get("request")
    children = [c for c in list(el) if isinstance(c.tag, str)]
    # Direct CDATA/text only counts as a template when there are no child
    # <header>/<body> elements (those belong to the built-in form).
    template = (el.text or "").strip() or None
    if children:
        template = None
    if method and (el.text or "").strip() and not children:
        raise ScenarioError("<send> cannot have both request= and a body template")
    if not method and not template:
        raise ScenarioError("<send> needs either request= or a CDATA template")
    if method:
        method = method.strip().upper()

    set_headers: dict[str, str] = {}
    remove_headers: list[str] = []
```

```
    body: str | None = None
    content_type: str | None = None
    for child in children:
        if child.tag == "header":
            name = child.get("name")
            if not name:
                raise ScenarioError("<header> requires a name attribute")
            action = (child.get("action") or "set").lower()
            if action == "remove":
                remove_headers.append(name)
            elif action == "set":
                set_headers[name] = (child.text or "").strip()
            else:
                raise ScenarioError(f"<header action> must be set|remove, got {action!r}")
        elif child.tag == "body":
            body = (child.text or "").strip()
            content_type = child.get("content-type") or child.get("content_type")
        else:
            raise ScenarioError(f"unsupported child <{child.tag}> inside <send>")

    if (set_headers or remove_headers or body is not None) and not method:
        raise ScenarioError("<header>/<body> require the built-in form (request=...)")

    return Send(
        builtin_method=method,
        template=template,
        retrans_ms=_int_attr(el, "retrans"),
        label=el.get("label"),
        set_headers=set_headers,
        remove_headers=remove_headers,
        body=body,
        content_type=content_type,
    )


def _parse_recv(el: ET.Element) -> Recv:
    response = _int_attr(el, "response")
    request = el.get("request")
    if response is None and not request:
        raise ScenarioError("<recv> needs response= or request=")
    if response is not None and not (100 <= response <= 699):
        raise ScenarioError(f"<recv response> must be 100-699, got {response}")
    actions: list[Extract] = []
    for action_el in el.findall("action"):
        for ereg in action_el.findall("ereg"):
            regexp = ereg.get("regexp")
            assign = ereg.get("assign_to") or ereg.get("variable")
            if not regexp or not assign:
                raise ScenarioError("<ereg> requires regexp and assign_to")
            actions.append(
                Extract(
                    regexp=regexp,
                    assign_to=assign.strip(),
                    search_in=ereg.get("search_in", "msg").lower(),
                    header=ereg.get("header"),
                    occurrence=_int_attr(ereg, "occurence") or 1,
                )
            )
    return Recv(
        response=response,
        request=request.strip().upper() if request else None,
        optional=_bool_attr(el, "optional"),
        timeout_ms=_int_attr(el, "timeout"),
        actions=actions,
        label=el.get("label"),
    )


def _validate_flow(steps: list[Step]) -> None:
    """Basic sanity checks on step ordering."""
    if not isinstance(steps[0], Send):
        raise ScenarioError("a UAC scenario must start with a <send>")


def parse_file(path: str | Path) -> Scenario:
    p = Path(path)
    if not p.is_file():
        raise ScenarioError(f"scenario file not found: {p}")
    return parse_string(p.read_text(encoding="utf-8"), name=p.name)
```

src/siptestwin/engine.py

Motor de ejecucion (UAC): construye y envia mensajes, empareja respuestas, maneja Digest, mantiene el dialogo para ACK/BYE, retransmite en UDP y registra metricas, traza y resultados por llamada. *(614 lineas)*

```
"""Scenario execution engine (UAC).

Drives a parsed scenario against the configured target: builds SIP requests,
sends them over the selected transport, matches responses to transactions,
handles Digest authentication, maintains the call dialog for ACK/BYE, performs
retransmissions on UDP, and records metrics and per-call outcomes.

Concurrency: a single shared transport is used; inbound messages are routed to
the owning call by Call-ID. Calls are paced by ``call_rate`` and limited by
``concurrency``.
"""

from __future__ import annotations

import queue
import socket
import threading
import time
from concurrent.futures import ThreadPoolExecutor
from dataclasses import dataclass, field

from . import sip_message as sip
from .authentication import build_authorization, parse_challenge
from .config import Config
from .dialog import Dialog
from .metrics import Metrics
from .reporting import CallRecord, RunResult
from .safety import RunLimits
from .scenario_parser import Pause, Recv, Scenario, Send
from .sip_message import SIPMessage
from .transaction import ClientTransaction, T1
from .transport import Transport, TransportError, create_transport


class ScenarioAssertionError(RuntimeError):
    """Raised when a received message does not satisfy a recv expectation."""


def _detect_local_ip(target_host: str, target_port: int) -> str:
    """Best-effort local source IP for Via/Contact, without sending data."""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            s.connect((target_host, target_port or 5060))
            return s.getsockname()[0]
        finally:
            s.close()
    except OSError:
        return "127.0.0.1"


@dataclass
class CallContext:
    index: int
    call_id: str
    from_tag: str
    inbox: "queue.Queue[SIPMessage]" = field(default_factory=queue.Queue)
    variables: dict[str, str] = field(default_factory=dict)
    cseq: int = 1
    last_request: SIPMessage | None = None
    last_invite: SIPMessage | None = None
    last_invite_response: SIPMessage | None = None
    dialog: Dialog | None = None
    current_tx: ClientTransaction | None = None
    auth_retried: bool = False
    steps_completed: int = 0


class Engine:
    def __init__(
        self,
        config: Config,
        scenario: Scenario,
        *,
        simulate: bool = False,
        metrics: Metrics | None = None,
        on_log=None,
        tracer=None,
    ) -> None:
        self.config = config
```

```
        self.scenario = scenario
        self.simulate = simulate
        self.metrics = metrics or Metrics()
        self.on_log = on_log or (lambda msg: None)
        self.tracer = tracer
        self.limits = RunLimits.from_config(config)

        ex = config.execution
        self.local_ip = ex.local_host or _detect_local_ip(config.target.host, config.target.port)
        self.local_port = ex.local_port
        self.transport_name = config.target.transport.lower()
        self.target_addr = (config.target.host, int(config.target.port))

        self._transport: Transport | None = None
        self._inboxes: dict[str, "queue.Queue[SIPMessage]"] = {}
        self._inbox_lock = threading.Lock()
        self._stop = threading.Event()

    # ----- lifecycle -----------------------------------------------------
    def run(self) -> RunResult:
        result = RunResult(
            target=f"{self.config.target.host}:{self.config.target.port}",
            scenario=self.scenario.name,
            transport=self.transport_name,
            simulate=self.simulate,
            started_at=time.time(),
            metrics=self.metrics,
        )
        if not self.simulate:
            self._start_transport()
        if self.tracer is not None:
            try:
                target_ip = socket.gethostbyname(self.config.target.host)
            except OSError:
                target_ip = self.config.target.host
            self.tracer.bind(self.local_ip, self.local_port, target_ip, self.transport_name)
        try:
            self._dispatch_calls(result)
        finally:
            self.local_port = self._bound_port()
            if self._transport is not None:
                self._transport.close()
            result.finished_at = time.time()
        return result

    def stop(self) -> None:
        self._stop.set()

    def _bound_port(self) -> int:
        if self._transport is not None:
            try:
                return self._transport.local_addr()[1]
            except Exception:
                return self.local_port
        return self.local_port

    def _start_transport(self) -> None:
        self._transport = create_transport(
            self.transport_name,
            peer=self.target_addr,
            bind_host="0.0.0.0",
            bind_port=self.local_port,
            verify_tls=self.config.security.verify_tls,
        )
        self._transport.start(self._on_inbound)
        self.local_port = self._transport.local_addr()[1]

    def _on_inbound(self, data: bytes, addr: tuple[str, int]) -> None:
        if self.tracer is not None:
            self.tracer.record("recv", data, addr)
        try:
            msg = sip.parse(data)
        except sip.SIPParseError:
            return
        call_id = msg.call_id
        if not call_id:
            return
        with self._inbox_lock:
            inbox = self._inboxes.get(call_id)
        if inbox is not None:
            inbox.put(msg)

    # ----- call scheduling -----------------------------------------------
    def _dispatch_calls(self, result: RunResult) -> None:
```

```
        total = self.limits.calls
        rate = self.limits.call_rate
        interval = 1.0 / rate if rate > 0 else 0.0
        deadline = (time.time() + self.limits.duration_seconds) if self.limits.duration_seconds else None
        records: list[CallRecord] = [None] * total  # type: ignore[list-item]

        with ThreadPoolExecutor(max_workers=self.limits.concurrency) as pool:
            futures = {}
            for i in range(total):
                if self._stop.is_set():
                    break
                if deadline and time.time() >= deadline:
                    break
                ctx = CallContext(
                    index=i,
                    call_id=sip.new_call_id(self.local_ip),
                    from_tag=sip.new_tag(),
                )
                with self._inbox_lock:
                    self._inboxes[ctx.call_id] = ctx.inbox
                futures[pool.submit(self._run_scenario, ctx)] = ctx
                if interval and i < total - 1:
                    time.sleep(interval)
            for fut, ctx in futures.items():
                try:
                    records[ctx.index] = fut.result()
                except Exception as exc:  # defensive: never lose a record
                    records[ctx.index] = CallRecord(
                        index=ctx.index, outcome="error", reason=str(exc)
                    )
                finally:
                    with self._inbox_lock:
                        self._inboxes.pop(ctx.call_id, None)

        result.calls = [r for r in records if r is not None]

    # ----- scenario execution ------------------------------------------
    def _run_scenario(self, ctx: CallContext) -> CallRecord:
        self.metrics.call_attempt()
        start = time.monotonic()
        final_code: int | None = None
        try:
            for step in self.scenario.steps:
                if self._stop.is_set():
                    raise ScenarioAssertionError("run stopped")
                if isinstance(step, Send):
                    self._do_send(ctx, step)
                elif isinstance(step, Recv):
                    code = self._do_recv(ctx, step)
                    if code is not None:
                        final_code = code
                elif isinstance(step, Pause):
                    self._do_pause(step)
                ctx.steps_completed += 1
            self.metrics.transaction_result(True)
            latency = (time.monotonic() - start) * 1000.0
            return CallRecord(
                index=ctx.index, outcome="passed", final_code=final_code,
                latency_ms=round(latency, 2), steps_completed=ctx.steps_completed,
            )
        except TimeoutError as exc:
            self.metrics.timeout()
            self.metrics.transaction_result(False, "timeout")
            return CallRecord(index=ctx.index, outcome="timeout", reason=str(exc),
                              final_code=final_code, steps_completed=ctx.steps_completed)
        except TransportError as exc:
            self.metrics.transport_error()
            self.metrics.transaction_result(False, "transport_error")
            return CallRecord(index=ctx.index, outcome="error", reason=str(exc),
                              final_code=final_code, steps_completed=ctx.steps_completed)
        except ScenarioAssertionError as exc:
            self.metrics.transaction_result(False, f"assertion:{exc}")
            return CallRecord(index=ctx.index, outcome="failed", reason=str(exc),
                              final_code=final_code, steps_completed=ctx.steps_completed)

    # ----- steps -------------------------------------------------------
    def _do_pause(self, step: Pause) -> None:
        if self.simulate:
            return
        # Cap any single pause so Ctrl+C stays responsive.
        remaining = step.milliseconds / 1000.0
        while remaining > 0 and not self._stop.is_set():
            chunk = min(remaining, 0.25)
            time.sleep(chunk)
```

```
            remaining -= chunk

    def _do_send(self, ctx: CallContext, step: Send) -> None:
        msg = self._build_message(ctx, step)
        ctx.last_request = msg
        if msg.method == "INVITE":
            ctx.last_invite = msg
            self.metrics.call_attempt  # noqa: B018 (attempt counted per scenario)
        branch = msg.top_via_branch() or sip.new_branch()
        reliable = self.transport_name in ("tcp", "tls")
        if msg.method not in ("ACK",):
            ctx.current_tx = ClientTransaction(
                request=msg, branch=branch, method=msg.method or "", reliable=reliable
            )
        self.on_log(f"[call {ctx.index}] >> {msg.method} {msg.request_uri}")
        if self.simulate:
            if self.tracer is not None:
                self.tracer.record("sent", msg.serialize(), self.target_addr)
            self.metrics.sent()
            return
        self._transmit(ctx, msg)

    def _transmit(self, ctx: CallContext, msg: SIPMessage) -> None:
        assert self._transport is not None
        data = msg.serialize()
        if self.tracer is not None:
            self.tracer.record("sent", data, self.target_addr)
        self._transport.send(data, self.target_addr)
        self.metrics.sent()

    def _do_recv(self, ctx: CallContext, step: Recv) -> int | None:
        if self.simulate:
            # Nothing is transmitted in simulate mode; treat as satisfied.
            return step.response
        timeout_s = (step.timeout_ms / 1000.0) if step.timeout_ms else self.config.execution.timeout_seconds
        deadline = time.monotonic() + timeout_s
        tx = ctx.current_tx

        while True:
            now = time.monotonic()
            if now >= deadline:
                if step.optional:
                    return None
                raise TimeoutError(
                    f"timed out waiting for {step.response or step.request} "
                    f"(call {ctx.index})"
                )
            # UDP retransmission while waiting.
            if tx is not None and tx.due_for_retransmit(now):
                self._transmit(ctx, tx.request)
                tx.note_retransmit(now)
                self.metrics.retransmit()

            try:
                msg = ctx.inbox.get(timeout=min(T1, max(0.0, deadline - now)))
            except queue.Empty:
                continue

            self.metrics.received(msg.status_code)
            if tx is not None and tx.matches(msg):
                tx.on_response(msg)

            code = msg.status_code or 0
            # Automatic Digest authentication. When a 401/407 arrives and auth
            # is enabled, answer the challenge with the configured credentials
            # and send the authenticated retry. If the scenario explicitly
            # expects the challenge code, this step is satisfied by it and the
            # authenticated response is matched by the following recv; otherwise
            # we keep waiting on this step for the post-auth code.
            if (
                msg.is_response
                and code in (401, 407)
                and self._auth_enabled()
                and not ctx.auth_retried
            ):
                self._auth_retry(ctx, msg)
                tx = ctx.current_tx
                deadline = time.monotonic() + timeout_s
                if step.response == code:
                    self._run_extractions(ctx, step, msg)
                    return code
                continue

            matched = self._message_matches(step, msg)
```

```
            if not matched:
                # Provisional/unrelated responses are expected noise while we
                # wait for the matching message; keep waiting.
                continue

            self._run_extractions(ctx, step, msg)

            if msg.is_response:
                if msg.method == "INVITE" or (ctx.last_invite and msg.cseq_method == "INVITE"):
                    self._handle_invite_response(ctx, msg)
                return code
            return None

    def _message_matches(self, step: Recv, msg: SIPMessage) -> bool:
        if step.response is not None:
            return msg.is_response and msg.status_code == step.response
        if step.request is not None:
            return msg.is_request and (msg.method or "").upper() == step.request
        return False

    def _handle_invite_response(self, ctx: CallContext, resp: SIPMessage) -> None:
        code = resp.status_code or 0
        if code >= 200 and code < 300 and ctx.last_invite is not None:
            ctx.last_invite_response = resp
            ctx.dialog = Dialog.from_invite_response(
                ctx.last_invite, resp,
                local_uri=self._from_uri(ctx),
                remote_uri=self._to_uri(),
            )
            self.metrics.call_established()

    # ----- message construction ----------------------------------------
    def _from_uri(self, ctx: CallContext) -> str:
        ident = self.config.identity
        uri = f"sip:{ident.from_user}@{ident.from_domain}"
        if ident.display_name:
            return f'"{ident.display_name}" <{uri}>'
        return f"<{uri}>"

    def _to_uri(self) -> str:
        ident = self.config.identity
        domain = ident.to_domain or self.config.target.host
        return f"<sip:{ident.to_user}@{domain}>"

    def _contact(self) -> str:
        ident = self.config.identity
        return f"<sip:{ident.contact_user}@{self.local_ip}:{self.local_port or 5060};transport={self.transport_
name}>"

    def _request_uri(self, method: str) -> str:
        ident = self.config.identity
        host = self.config.target.host
        if method == "REGISTER":
            return f"sip:{host}"
        domain = ident.to_domain or host
        return f"sip:{ident.to_user}@{domain}"

    def _build_message(self, ctx: CallContext, step: Send) -> SIPMessage:
        if step.template is not None:
            return self._build_from_template(ctx, step.template)
        method = step.builtin_method or ""
        if method == "ACK":
            return self._build_ack(ctx)
        if method == "BYE":
            return self._build_bye(ctx)
        return self._build_builtin(ctx, method, step)

    def _build_builtin(self, ctx: CallContext, method: str, step: Send | None = None) -> SIPMessage:
        mc = self.config.message
        extra: dict[str, str] = {}
        if method in ("INVITE", "REGISTER", "SUBSCRIBE", "OPTIONS"):
            extra["Allow"] = "INVITE, ACK, CANCEL, OPTIONS, BYE"
        if method == "REGISTER":
            extra["Expires"] = "3600"

        # Body resolution: scenario <body> > config message.invite.sdp >
        # generated SDP (INVITE only). Any method may carry a custom body.
        body, content_type = self._resolve_body(ctx, method, step)
        if content_type:
            extra["Content-Type"] = content_type

        contact = self._contact() if method in ("INVITE", "REGISTER", "SUBSCRIBE") else None
        msg = sip.build_request(
            method,
```

```
            self._request_uri(method),
            from_uri=self._from_uri(ctx),
            to_uri=self._to_uri() if method != "REGISTER" else self._from_uri(ctx),
            call_id=ctx.call_id,
            cseq=ctx.cseq,
            via_host=self.local_ip,
            via_port=self.local_port or 5060,
            transport=self.transport_name.upper(),
            from_tag=ctx.from_tag,
            contact=contact,
            extra_headers=extra,
            body=body,
        )
        ctx.cseq += 1
        self._apply_header_overrides(ctx, msg, method, step)
        return msg

    def _resolve_body(self, ctx: CallContext, method: str,
                      step: Send | None) -> tuple[bytes, str | None]:
        mc = self.config.message
        if step is not None and step.body is not None:
            ct = step.content_type or (mc.invite.content_type if method == "INVITE" else None)
            return self._substitute(ctx, step.body).encode("utf-8"), ct
        if method == "INVITE":
            if mc.invite.sdp:
                return self._substitute(ctx, mc.invite.sdp).encode("utf-8"), mc.invite.content_type
            sdp = sip.generate_sdp(self.local_ip, session_id=sip.process_id() + ctx.index)
            return sdp, mc.invite.content_type
        return b"", None

    def _apply_header_overrides(self, ctx: CallContext, msg: SIPMessage,
                                method: str, step: Send | None) -> None:
        """Apply configured and per-send header customizations.

        Precedence (low to high): config.message.extra_headers, then
        config.message.invite.extra_headers (INVITE), then the scenario's
        <header> set. Removals (config then scenario) run last. Values support
        scenario placeholders. This is also how malformed-header robustness
        cases are expressed (e.g. removing a mandatory header on purpose).
        """
        mc = self.config.message
        for name, val in mc.extra_headers.items():
            msg.headers.set(name, self._substitute(ctx, str(val)))
        if method == "INVITE":
            for name, val in mc.invite.extra_headers.items():
                msg.headers.set(name, self._substitute(ctx, str(val)))
        if step is not None:
            for name, val in step.set_headers.items():
                msg.headers.set(name, self._substitute(ctx, str(val)))
        for name in mc.remove_headers:
            msg.headers.remove(name)
        if step is not None:
            for name in step.remove_headers:
                msg.headers.remove(name)
        msg.headers.set("Content-Length", str(len(msg.body)))

    def _build_ack(self, ctx: CallContext) -> SIPMessage:
        if ctx.dialog is None or ctx.last_invite is None or ctx.last_invite_response is None:
            raise ScenarioAssertionError("ACK requested but no established INVITE dialog")
        return ctx.dialog.build_ack(
            ctx.last_invite, ctx.last_invite_response,
            self.local_ip, self.local_port or 5060, self.transport_name.upper(),
        )

    def _build_bye(self, ctx: CallContext) -> SIPMessage:
        if ctx.dialog is None:
            raise ScenarioAssertionError("BYE requested but no established dialog")
        bye = ctx.dialog.build_bye(self.local_ip, self.local_port or 5060,
                                   self.transport_name.upper())
        self.metrics.call_terminated()
        return bye

    def _build_from_template(self, ctx: CallContext, template: str) -> SIPMessage:
        text = self._substitute(ctx, template)
        msg = sip.parse(text.encode("utf-8"))
        if msg.method:
            ctx.cseq = (msg.cseq_number or ctx.cseq) + 1
        return msg

    def _substitute(self, ctx: CallContext, template: str) -> str:
        ident = self.config.identity
        branch = sip.new_branch()
        mapping = {
            "[call_id]": ctx.call_id,
```

```
            "[branch]": branch,
            "[cseq]": str(ctx.cseq),
            "[local_ip]": self.local_ip,
            "[local_port]": str(self.local_port or 5060),
            "[remote_ip]": self.config.target.host,
            "[remote_port]": str(self.config.target.port),
            "[transport]": self.transport_name.upper(),
            "[from_user]": ident.from_user,
            "[from_domain]": ident.from_domain,
            "[from_tag]": ctx.from_tag,
            "[to_user]": ident.to_user,
            "[to_domain]": ident.to_domain or self.config.target.host,
            "[contact_user]": ident.contact_user,
            "[service]": ident.to_user,
            "[len]": "0",
        }
        out = template.replace("\n", "\r\n").replace("\r\r\n", "\r\n")
        for key, val in mapping.items():
            out = out.replace(key, val)
        for name, val in ctx.variables.items():
            out = out.replace(f"[${name}]", val)
        return out

    # ----- extraction + auth -----------------------------------------------
    def _run_extractions(self, ctx: CallContext, step: Recv, msg: SIPMessage) -> None:
        for extract in step.actions:
            pattern = extract.compile()
            if extract.search_in == "hdr" and extract.header:
                haystack = "\n".join(msg.headers.get_all(extract.header))
            else:
                haystack = msg.serialize().decode("utf-8", errors="replace")
            found = pattern.findall(haystack)
            if found:
                value = found[min(extract.occurrence - 1, len(found) - 1)]
                if isinstance(value, tuple):
                    value = value[0]
                ctx.variables[extract.assign_to] = value

    def _auth_enabled(self) -> bool:
        return self.config.security.authentication.lower() == "digest"

    def _auth_retry(self, ctx: CallContext, resp: SIPMessage) -> None:
        if ctx.last_request is None:
            raise ScenarioAssertionError("auth challenge with no prior request")
        is_proxy = resp.status_code == 407
        chal_header = "Proxy-Authenticate" if is_proxy else "WWW-Authenticate"
        raw = resp.headers.get(chal_header)
        if not raw:
            raise ScenarioAssertionError(f"{resp.status_code} without {chal_header}")
        challenge = parse_challenge(raw, is_proxy=is_proxy)

        # For an INVITE challenge we must ACK the failure response first.
        if (ctx.last_request.method == "INVITE"):
            self._ack_failure(ctx, resp)

        username = self.config.security.username or self.config.identity.from_user
        password = self.config.security.password or ""
        orig = ctx.last_request
        method = orig.method or ""
        uri = orig.request_uri or ""
        auth_value = build_authorization(
            challenge, username=username, password=password, method=method, uri=uri
        )
        header_name = "Proxy-Authorization" if is_proxy else "Authorization"

        retry = sip.build_request(
            method,
            uri,
            from_uri=self._from_uri(ctx),
            to_uri=orig.headers.get("To", self._to_uri()).split(";tag=")[0],
            call_id=ctx.call_id,
            cseq=ctx.cseq,
            via_host=self.local_ip,
            via_port=self.local_port or 5060,
            transport=self.transport_name.upper(),
            from_tag=ctx.from_tag,
            contact=self._contact() if method in ("INVITE", "REGISTER") else None,
            extra_headers={header_name: auth_value,
                           **({"Content-Type": "application/sdp"} if orig.body else {})},
            body=orig.body,
        )
        ctx.cseq += 1
        ctx.auth_retried = True
        ctx.last_request = retry
```

```
        if method == "INVITE":
            ctx.last_invite = retry
        reliable = self.transport_name in ("tcp", "tls")
        ctx.current_tx = ClientTransaction(
            request=retry, branch=retry.top_via_branch() or sip.new_branch(),
            method=method, reliable=reliable,
        )
        self.on_log(f"[call {ctx.index}] >> {method} (authenticated retry)")
        self._transmit(ctx, retry)

    def _ack_failure(self, ctx: CallContext, resp: SIPMessage) -> None:
        """ACK a non-2xx INVITE response (same branch, RFC 3261 17.1.1.3)."""
        if ctx.last_invite is None:
            return
        inv = ctx.last_invite
        ack = sip.build_request(
            "ACK",
            inv.request_uri or "",
            from_uri=inv.headers.get("From", ""),
            to_uri=resp.headers.get("To", inv.headers.get("To", "")),
            call_id=ctx.call_id,
            cseq=inv.cseq_number or ctx.cseq,
            via_host=self.local_ip,
            via_port=self.local_port or 5060,
            transport=self.transport_name.upper(),
            branch=inv.top_via_branch(),
        )
        # ACK re-uses the INVITE's From (with tag) and the response's To (with tag).
        ack.headers.set("From", inv.headers.get("From", ""))
        ack.headers.set("To", resp.headers.get("To", inv.headers.get("To", "")))
        self._transmit(ctx, ack)
```

src/siptestwin/metrics.py

Metricas thread-safe: contadores, categorias de respuesta, percentiles de latencia (p50/p95/p99) y motivos de fallo. *(133 lineas)*

```
"""Metrics collection: counters, latency percentiles and failure reasons.

Local generator errors (timeouts, transport failures, scenario assertion
failures) are tracked separately from server-reported SIP failures so reports
can distinguish "the server rejected us" from "our test runner had a problem".
"""

from __future__ import annotations

import threading
from collections import Counter
from dataclasses import dataclass, field


def _percentile(sorted_values: list[float], pct: float) -> float:
    if not sorted_values:
        return 0.0
    if len(sorted_values) == 1:
        return sorted_values[0]
    rank = (pct / 100.0) * (len(sorted_values) - 1)
    lo = int(rank)
    hi = min(lo + 1, len(sorted_values) - 1)
    frac = rank - lo
    return sorted_values[lo] + (sorted_values[hi] - sorted_values[lo]) * frac


@dataclass
class Metrics:
    """Thread-safe aggregate metrics for a run."""

    messages_sent: int = 0
    messages_received: int = 0
    retransmissions: int = 0
    transport_errors: int = 0
    timeouts: int = 0

    calls_attempted: int = 0
    calls_established: int = 0
    calls_terminated: int = 0

    transactions_completed: int = 0
    transactions_failed: int = 0

    response_codes: Counter = field(default_factory=Counter)
    failure_reasons: Counter = field(default_factory=Counter)
```

```
    _latencies_ms: list[float] = field(default_factory=list)
    _lock: threading.Lock = field(default_factory=threading.Lock, repr=False)

    # ----- increments -----------------------------------------------------
    def sent(self, n: int = 1) -> None:
        with self._lock:
            self.messages_sent += n

    def received(self, code: int | None = None) -> None:
        with self._lock:
            self.messages_received += 1
            if code is not None:
                self.response_codes[code] += 1

    def retransmit(self) -> None:
        with self._lock:
            self.retransmissions += 1

    def transport_error(self) -> None:
        with self._lock:
            self.transport_errors += 1

    def timeout(self) -> None:
        with self._lock:
            self.timeouts += 1

    def latency(self, ms: float) -> None:
        with self._lock:
            self._latencies_ms.append(ms)

    def call_attempt(self) -> None:
        with self._lock:
            self.calls_attempted += 1

    def call_established(self) -> None:
        with self._lock:
            self.calls_established += 1

    def call_terminated(self) -> None:
        with self._lock:
            self.calls_terminated += 1

    def transaction_result(self, ok: bool, reason: str | None = None) -> None:
        with self._lock:
            if ok:
                self.transactions_completed += 1
            else:
                self.transactions_failed += 1
                if reason:
                    self.failure_reasons[reason] += 1

    # ----- derived --------------------------------------------------------
    def latency_percentiles(self) -> dict[str, float]:
        with self._lock:
            vals = sorted(self._latencies_ms)
        return {
            "count": len(vals),
            "min_ms": round(vals[0], 2) if vals else 0.0,
            "p50_ms": round(_percentile(vals, 50), 2),
            "p95_ms": round(_percentile(vals, 95), 2),
            "p99_ms": round(_percentile(vals, 99), 2),
            "max_ms": round(vals[-1], 2) if vals else 0.0,
        }

    def response_by_category(self) -> dict[str, int]:
        cats: Counter = Counter()
        with self._lock:
            for code, n in self.response_codes.items():
                cats[f"{code // 100}xx"] += n
        return dict(sorted(cats.items()))

    def to_dict(self) -> dict:
        return {
            "messages_sent": self.messages_sent,
            "messages_received": self.messages_received,
            "retransmissions": self.retransmissions,
            "transport_errors": self.transport_errors,
            "timeouts": self.timeouts,
            "calls_attempted": self.calls_attempted,
            "calls_established": self.calls_established,
            "calls_terminated": self.calls_terminated,
            "transactions_completed": self.transactions_completed,
            "transactions_failed": self.transactions_failed,
            "response_codes": {str(k): v for k, v in sorted(self.response_codes.items())},
```

```
            "response_by_category": self.response_by_category(),
            "failure_reasons": dict(self.failure_reasons),
            "latency": self.latency_percentiles(),
        }
```

src/siptestwin/reporting.py

Generacion de reportes JSON/CSV y resumen en consola (Rich). *(180 lineas)*

```
"""Report generation: JSON, CSV and a human-readable console summary.

Reports never contain credential material (see :func:`siptestwin.safety.redact`).
"""

from __future__ import annotations

import csv
import json
import time
from dataclasses import dataclass, field
from pathlib import Path

from rich.console import Console
from rich.table import Table

from .metrics import Metrics


@dataclass
class CallRecord:
    """Outcome of a single scenario execution (one call/transaction run)."""

    index: int
    outcome: str                # "passed" | "failed" | "timeout" | "error"
    final_code: int | None = None
    reason: str | None = None
    latency_ms: float | None = None
    steps_completed: int = 0


@dataclass
class RunResult:
    target: str
    scenario: str
    transport: str
    simulate: bool
    started_at: float
    finished_at: float = 0.0
    metrics: Metrics = field(default_factory=Metrics)
    calls: list[CallRecord] = field(default_factory=list)
    config_summary: dict = field(default_factory=dict)

    @property
    def duration_s(self) -> float:
        return round((self.finished_at or time.time()) - self.started_at, 3)

    @property
    def passed(self) -> int:
        return sum(1 for c in self.calls if c.outcome == "passed")

    @property
    def failed(self) -> int:
        return sum(1 for c in self.calls if c.outcome != "passed")

    def to_dict(self) -> dict:
        return {
            "tool": "SIPTestWin",
            "target": self.target,
            "scenario": self.scenario,
            "transport": self.transport,
            "mode": "simulate" if self.simulate else "active",
            "started_at": time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(self.started_at)),
            "duration_seconds": self.duration_s,
            "summary": {
                "calls_total": len(self.calls),
                "passed": self.passed,
                "failed": self.failed,
            },
            "metrics": self.metrics.to_dict(),
            "calls": [vars(c) for c in self.calls],
            "config": self.config_summary,
```

```
        }
| |
| |
| | def write_reports(result: RunResult, output_dir: str | Path, formats: list[str]) -> list[Path]:
| |     out = Path(output_dir)
| |     out.mkdir(parents=True, exist_ok=True)
| |     stamp = time.strftime("%Y%m%d-%H%M%S", time.localtime(result.started_at))
| |     written: list[Path] = []
| |
| |     if "json" in [f.lower() for f in formats]:
| |         p = out / f"report-{stamp}.json"
| |         p.write_text(json.dumps(result.to_dict(), indent=2, ensure_ascii=False), encoding="utf-8")
| |         written.append(p)
| |
| |     if "csv" in [f.lower() for f in formats]:
| |         p = out / f"calls-{stamp}.csv"
| |         with p.open("w", newline="", encoding="utf-8") as fh:
| |             writer = csv.writer(fh)
| |             writer.writerow(["index", "outcome", "final_code", "reason",
| |                              "latency_ms", "steps_completed"])
| |             for c in result.calls:
| |                 writer.writerow([c.index, c.outcome, c.final_code or "",
| |                                  c.reason or "", c.latency_ms or "", c.steps_completed])
| |         written.append(p)
| |
| |         pm = out / f"metrics-{stamp}.csv"
| |         with pm.open("w", newline="", encoding="utf-8") as fh:
| |             writer = csv.writer(fh)
| |             writer.writerow(["metric", "value"])
| |             flat = result.metrics.to_dict()
| |             for key in ("messages_sent", "messages_received", "retransmissions",
| |                         "transport_errors", "timeouts", "calls_attempted",
| |                         "calls_established", "calls_terminated",
| |                         "transactions_completed", "transactions_failed"):
| |                 writer.writerow([key, flat[key]])
| |             for cat, n in flat["response_by_category"].items():
| |                 writer.writerow([f"responses_{cat}", n])
| |             for k, v in flat["latency"].items():
| |                 writer.writerow([f"latency_{k}", v])
| |         written.append(pm)
| |
| |     return written
| |
| |
| | def print_summary(result: RunResult, console: Console | None = None) -> None:
| |     console = console or Console()
| |     m = result.metrics.to_dict()
| |
| |     header = Table.grid(padding=(0, 2))
| |     header.add_column(style="bold")
| |     header.add_column()
| |     header.add_row("Target", result.target)
| |     header.add_row("Scenario", result.scenario)
| |     header.add_row("Transport", result.transport.upper())
| |     header.add_row("Mode", "SIMULATE (no packets sent)" if result.simulate else "ACTIVE")
| |     header.add_row("Duration", f"{result.duration_s}s")
| |     console.print(header)
| |
| |     summary = Table(title="Result", show_header=True, header_style="bold")
| |     summary.add_column("Metric")
| |     summary.add_column("Value", justify="right")
| |     summary.add_row("Calls total", str(len(result.calls)))
| |     summary.add_row("[green]Passed[/green]", str(result.passed))
| |     summary.add_row("[red]Failed[/red]", str(result.failed))
| |     summary.add_row("Messages sent / received",
| |                     f"{m['messages_sent']} / {m['messages_received']}")
| |     summary.add_row("Retransmissions", str(m["retransmissions"]))
| |     summary.add_row("Timeouts", str(m["timeouts"]))
| |     summary.add_row("Transport errors", str(m["transport_errors"]))
| |     summary.add_row("Calls established / terminated",
| |                     f"{m['calls_established']} / {m['calls_terminated']}")
| |     console.print(summary)
| |
| |     if m["response_by_category"]:
| |         rc = Table(title="Responses by category", header_style="bold")
| |         rc.add_column("Category")
| |         rc.add_column("Count", justify="right")
| |         for cat, n in m["response_by_category"].items():
| |             rc.add_row(cat, str(n))
| |         console.print(rc)
| |
| |     lat = m["latency"]
| |     if lat["count"]:
| |         lt = Table(title="Latency (ms)", header_style="bold")
```

```
        for k in ("min_ms", "p50_ms", "p95_ms", "p99_ms", "max_ms"):
            lt.add_column(k.replace("_ms", ""), justify="right")
        lt.add_row(*[str(lat[k]) for k in ("min_ms", "p50_ms", "p95_ms", "p99_ms", "max_ms")])
        console.print(lt)

    if m["failure_reasons"]:
        fr = Table(title="Failure reasons", header_style="bold")
        fr.add_column("Reason")
        fr.add_column("Count", justify="right")
        for reason, n in m["failure_reasons"].items():
            fr.add_row(reason, str(n))
        console.print(fr)


def load_reports(input_dir: str | Path) -> list[dict]:
    """Load previously written JSON reports from a directory."""
    out = Path(input_dir)
    reports = []
    for p in sorted(out.glob("report-*.json")):
        try:
            reports.append(json.loads(p.read_text(encoding="utf-8")))
        except (json.JSONDecodeError, OSError):
            continue
    return reports
```

src/siptestwin/capture.py

Traza/captura de mensajes SIP: consola, .trace.log de texto y .pcap abrible en Wireshark (cabeceras Ethernet/IPv4/UDP sinteticas). *(193 lineas)*

```
"""SIP message tracing and capture.

Provides a thread-safe tracer that records every SIP message the engine sends
or receives, in up to three sinks:

* **console** — a compact one-line-per-message ladder (with the first line of
  each SIP message), optionally with the full message;
* **text** — a human-readable ``.trace.log`` with timestamps and full messages
  (credential hashes redacted, consistent with the audit policy);
* **pcap** — a Wireshark-openable capture. Each SIP message is wrapped in
  synthesized Ethernet/IPv4/UDP headers so Wireshark dissects it as SIP. This
  is a true wire capture: SIP Digest never transmits the plaintext password,
  so no plaintext secret can appear here.

No external tools (tshark/npcap) are required to produce the pcap.
"""

from __future__ import annotations

import socket
import struct
import threading
import time
from pathlib import Path

from .safety import redact

_LINKTYPE_ETHERNET = 1


def _resolve_ipv4(host: str) -> bytes:
    try:
        return socket.inet_aton(socket.gethostbyname(host))
    except (OSError, ValueError):
        return socket.inet_aton("0.0.0.0")


def _ip_checksum(data: bytes) -> int:
    if len(data) % 2:
        data += b"\x00"
    total = 0
    for i in range(0, len(data), 2):
        total += (data[i] << 8) + data[i + 1]
    total = (total >> 16) + (total & 0xFFFF)
    total += total >> 16
    return ~total & 0xFFFF


def _synthesize_udp_packet(src_ip: str, src_port: int, dst_ip: str,
                           dst_port: int, payload: bytes) -> bytes:
```

```
    """Build an Ethernet+IPv4+UDP frame carrying ``payload`` (for pcap)."""
    # Ethernet header: locally-administered placeholder MACs, IPv4 ethertype.
    eth = b"\x02\x00\x00\x00\x00\x02" + b"\x02\x00\x00\x00\x00\x01" + b"\x08\x00"

    udp_len = 8 + len(payload)
    udp = struct.pack("!HHHH", src_port & 0xFFFF, dst_port & 0xFFFF, udp_len, 0) + payload

    total_len = 20 + udp_len
    src = _resolve_ipv4(src_ip)
    dst = _resolve_ipv4(dst_ip)
    ip_no_csum = struct.pack(
        "!BBHHHBBH", 0x45, 0x00, total_len, 0x0000, 0x4000, 64, 17, 0
    ) + src + dst
    csum = _ip_checksum(ip_no_csum)
    ip = ip_no_csum[:10] + struct.pack("!H", csum) + ip_no_csum[12:]
    return eth + ip + udp


class PcapWriter:
    """Minimal classic-format (libpcap) writer, Ethernet link type."""

    def __init__(self, path: str | Path) -> None:
        self.path = Path(path)
        self.path.parent.mkdir(parents=True, exist_ok=True)
        self._fh = self.path.open("wb")
        # Global header.
        self._fh.write(struct.pack("!IHHiIII", 0xA1B2C3D4, 2, 4, 0, 0, 65535,
                                   _LINKTYPE_ETHERNET))
        self._fh.flush()

    def write(self, frame: bytes, ts: float) -> None:
        sec = int(ts)
        usec = int((ts - sec) * 1_000_000)
        self._fh.write(struct.pack("!IIII", sec, usec, len(frame), len(frame)))
        self._fh.write(frame)
        self._fh.flush()

    def close(self) -> None:
        try:
            self._fh.close()
        except OSError:
            pass


class SipTracer:
    """Records sent/received SIP messages to console/text/pcap sinks."""

    def __init__(
        self,
        *,
        to_console: bool = False,
        text_path: str | Path | None = None,
        pcap_path: str | Path | None = None,
        console_full: bool = False,
        console_print=None,
    ) -> None:
        self.to_console = to_console
        self.console_full = console_full
        self._print = console_print or print
        self._lock = threading.Lock()
        self._count = 0

        self.text_path = str(text_path) if text_path else None
        self.pcap_path = str(pcap_path) if pcap_path else None

        self._text = None
        if text_path:
            p = Path(text_path)
            p.parent.mkdir(parents=True, exist_ok=True)
            self._text = p.open("w", encoding="utf-8")
            self._text.write(f"# SIPTestWin SIP trace - {time.strftime('%Y-%m-%d %H:%M:%S')}\n\n")

        self._pcap = PcapWriter(pcap_path) if pcap_path else None

        # Bound by the engine once the transport local address is known.
        self.local_ip = "0.0.0.0"
        self.local_port = 0
        self.target_ip = "0.0.0.0"
        self.transport = "udp"

    def bind(self, local_ip: str, local_port: int, target_ip: str, transport: str) -> None:
        self.local_ip = local_ip
        self.local_port = local_port or 0
        self.target_ip = target_ip
```

```
        self.transport = transport

    @property
    def enabled(self) -> bool:
        return self.to_console or self._text is not None or self._pcap is not None

    def record(self, direction: str, data: bytes, peer: tuple[str, int]) -> None:
        """direction: 'sent' or 'recv'. ``peer`` is the remote (host, port)."""
        ts = time.time()
        first_line = data.split(b"\r\n", 1)[0].decode("utf-8", errors="replace")
        arrow = "->" if direction == "sent" else "<-"
        peer_ip, peer_port = peer[0], peer[1]

        with self._lock:
            self._count += 1
            seq = self._count

            if self.to_console:
                stamp = time.strftime("%H:%M:%S", time.localtime(ts))
                self._print(f"[trace {seq:>3}] {stamp} {arrow} {peer_ip}:{peer_port}  {first_line}")
                if self.console_full:
                    self._print(redact(data.decode("utf-8", errors="replace")))

            if self._text is not None:
                stamp = time.strftime("%Y-%m-%d %H:%M:%S", time.localtime(ts))
                hdr = (f"===== #{seq} {direction.upper()} {arrow} {peer_ip}:{peer_port} "
                       f"({self.transport.upper()}) {stamp} =====\n")
                body = redact(data.decode("utf-8", errors="replace"))
                self._text.write(hdr + body.rstrip("\r\n") + "\n\n")
                self._text.flush()

            if self._pcap is not None:
                if direction == "sent":
                    frame = _synthesize_udp_packet(self.local_ip, self.local_port,
                                                   peer_ip, peer_port, data)
                else:
                    frame = _synthesize_udp_packet(peer_ip, peer_port,
                                                   self.local_ip, self.local_port, data)
                self._pcap.write(frame, ts)

    def close(self) -> None:
        with self._lock:
            if self._text is not None:
                try:
                    self._text.close()
                except OSError:
                    pass
                self._text = None
            if self._pcap is not None:
                self._pcap.close()
                self._pcap = None

    def paths(self) -> list[str]:
        out = []
        if self.text_path:
            out.append(self.text_path)
        if self.pcap_path:
            out.append(self.pcap_path)
        return out
```

src/siptestwin/netutils.py

Utilidades de red: enumera las IPv4 locales disponibles para elegir la IP de origen (comando interfaces y --local-host). *(94 lineas)*

```
"""Local network helpers: enumerate usable source IPv4 addresses.

Used by the ``interfaces`` CLI command and to help the operator choose which
local IP to trace / bind on (``execution.local_host``). Pure standard library,
no external dependencies, works on Windows.
"""

from __future__ import annotations

import ipaddress
import socket
from dataclasses import dataclass


@dataclass
```

```
class LocalAddress:
    ip: str
    label: str          # "loopback", "private", "public", "link-local"
    is_loopback: bool


def _classify(ip: str) -> tuple[str, bool]:
    try:
        addr = ipaddress.ip_address(ip)
    except ValueError:
        return "unknown", False
    if addr.is_loopback:
        return "loopback", True
    if addr.is_link_local:
        return "link-local", False
    if addr.is_private:
        return "private", False
    return "public", False


def list_local_ipv4() -> list[LocalAddress]:
    """Return the distinct IPv4 addresses this host can bind as a source.

    Combines the hostname resolution and the "connect a UDP socket to a public
    address and read the chosen source" trick (which reveals the default-route
    interface IP) without sending any packets.
    """
    found: dict[str, LocalAddress] = {}

    def add(ip: str) -> None:
        if not ip or ip in found:
            return
        label, loop = _classify(ip)
        found[ip] = LocalAddress(ip=ip, label=label, is_loopback=loop)

    # Always include loopback (used for local lab testing).
    add("127.0.0.1")

    # Hostname-resolved addresses.
    try:
        hostname = socket.gethostname()
        for info in socket.getaddrinfo(hostname, None, socket.AF_INET):
            add(info[4][0])
    except (socket.gaierror, OSError):
        pass

    # Default-route source IP (no packet is actually sent for UDP connect).
    for probe in ("8.8.8.8", "192.0.2.1"):
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            try:
                s.connect((probe, 9))
                add(s.getsockname()[0])
            finally:
                s.close()
        except OSError:
            continue

    # Order: non-loopback private first, then public, link-local, loopback last.
    order = {"private": 0, "public": 1, "link-local": 2, "unknown": 3, "loopback": 4}
    return sorted(found.values(), key=lambda a: (order.get(a.label, 3), a.ip))


def default_local_ip(target_host: str = "8.8.8.8", target_port: int = 9) -> str:
    """Best-effort default source IP toward ``target_host`` (no packet sent)."""
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        try:
            s.connect((target_host, target_port))
            return s.getsockname()[0]
        finally:
            s.close()
    except OSError:
        return "127.0.0.1"


def is_local_ip(ip: str) -> bool:
    """True if ``ip`` is one of this host's bindable IPv4 addresses."""
    return any(a.ip == ip for a in list_local_ipv4())
```

src/siptestwin/security_tests/__init__.py

Paquete de pruebas de seguridad/robustez. *(22 lineas)*

```
"""Independent SIP security / robustness test modules.

Each module checks one property of a target's behaviour. All tests are bounded
and non-destructive: they validate protocol conformance, authentication
challenge handling, and controlled handling of malformed input. There is no
credential brute-forcing, unbounded flooding, or automated third-party
exploitation here by design.
"""

from .malformed import MalformedCase, malformed_catalog
from .protocol import ProtocolFinding, validate_message, validate_response_sequence
from .regression import RegressionResult, compare_to_baseline

__all__ = [
    "MalformedCase",
    "malformed_catalog",
    "ProtocolFinding",
    "validate_message",
    "validate_response_sequence",
    "RegressionResult",
    "compare_to_baseline",
]
```

src/siptestwin/security_tests/protocol.py

Validacion de conformidad de mensajes y secuencias de respuesta. *(93 lineas)*

```
"""Protocol-validation checks for SIP messages and response sequences."""

from __future__ import annotations

from dataclasses import dataclass

from ..sip_message import SIPMessage

# Mandatory headers in any SIP request (RFC 3261 section 8.1.1).
MANDATORY_REQUEST_HEADERS = ("Via", "From", "To", "Call-ID", "CSeq", "Max-Forwards")
MANDATORY_RESPONSE_HEADERS = ("Via", "From", "To", "Call-ID", "CSeq")


@dataclass
class ProtocolFinding:
    severity: str   # "error" | "warning" | "info"
    rule: str
    detail: str


def validate_message(msg: SIPMessage) -> list[ProtocolFinding]:
    """Validate structural conformance of a request or response."""
    findings: list[ProtocolFinding] = []
    required = MANDATORY_REQUEST_HEADERS if msg.is_request else MANDATORY_RESPONSE_HEADERS
    for header in required:
        if header not in msg.headers:
            findings.append(ProtocolFinding("error", "missing-header",
                                            f"mandatory header {header} is absent"))

    # These headers must appear exactly once (RFC 3261 section 8.1.1 / 20).
    for header in ("From", "To", "Call-ID", "CSeq", "Max-Forwards"):
        if len(msg.headers.get_all(header)) > 1:
            findings.append(ProtocolFinding("error", "duplicate-header",
                                            f"{header} appears more than once"))

    cseq = msg.headers.get("CSeq")
    if cseq:
        parts = cseq.split()
        if len(parts) != 2 or not parts[0].isdigit():
            findings.append(ProtocolFinding("error", "bad-cseq", f"malformed CSeq: {cseq!r}"))
        elif msg.is_request and parts[1].upper() != (msg.method or "").upper():
            findings.append(ProtocolFinding("error", "cseq-method-mismatch",
                                            f"CSeq method {parts[1]} != {msg.method}"))

    via = msg.headers.get("Via")
    if via and "branch=" not in via:
        findings.append(ProtocolFinding("error", "via-no-branch",
                                        "top Via lacks a branch parameter"))
    if via and "z9hG4bK" not in (via.split("branch=")[-1] if "branch=" in via else ""):
        findings.append(ProtocolFinding("warning", "via-branch-cookie",
                                        "Via branch missing RFC 3261 magic cookie"))

```

```
    if msg.is_request and msg.headers.get("Max-Forwards"):
        try:
            mf = int(msg.headers.get("Max-Forwards"))
            if mf < 0:
                findings.append(ProtocolFinding("error", "bad-max-forwards",
                                                "Max-Forwards is negative"))
        except ValueError:
            findings.append(ProtocolFinding("error", "bad-max-forwards",
                                            "Max-Forwards is not an integer"))

    cl = msg.headers.get("Content-Length")
    if cl is not None:
        try:
            if int(cl) != len(msg.body):
                findings.append(ProtocolFinding("warning", "content-length-mismatch",
                                                f"Content-Length {cl} != body {len(msg.body)}"))
        except ValueError:
            findings.append(ProtocolFinding("error", "bad-content-length",
                                            f"Content-Length not an integer: {cl!r}"))
    return findings


def validate_response_sequence(codes: list[int]) -> list[ProtocolFinding]:
    """Validate that a sequence of response codes for a request is plausible."""
    findings: list[ProtocolFinding] = []
    if not codes:
        findings.append(ProtocolFinding("warning", "no-response", "no responses received"))
        return findings
    finals = [c for c in codes if c >= 200]
    if len(finals) == 0:
        findings.append(ProtocolFinding("warning", "no-final",
                                        "only provisional responses received"))
    # Provisional after final is a protocol violation.
    seen_final = False
    for c in codes:
        if c >= 200:
            seen_final = True
        elif seen_final:
            findings.append(ProtocolFinding("error", "provisional-after-final",
                                            f"provisional {c} after a final response"))
    return findings
```

src/siptestwin/security_tests/malformed.py

Catalogo fijo y acotado de peticiones malformadas para comprobar que el servidor las rechaza con gracia (no es un fuzzer). *(86 lineas)*

```
"""Controlled malformed-input catalog for robustness testing.

This produces a *small, fixed* set of individually malformed requests to check
that a server rejects them gracefully (typically with a 4xx) rather than
crashing. It is deliberately bounded: one message per case, no flooding, no
fuzzing at volume. Each case documents what it violates and the expected
graceful handling.
"""

from __future__ import annotations

from dataclasses import dataclass

from .. import sip_message as sip
from ..sip_message import SIPMessage


@dataclass
class MalformedCase:
    name: str
    description: str
    expected: str                # description of acceptable server behaviour
    message: SIPMessage


def _base_options(local_ip: str, target_host: str, target_port: int) -> SIPMessage:
    return sip.build_request(
        "OPTIONS",
        f"sip:{target_host}",
        from_uri="<sip:tester@lab.example>",
        to_uri=f"<sip:probe@{target_host}>",
        call_id=sip.new_call_id(local_ip),
        cseq=1,
```

```
        via_host=local_ip,
        via_port=target_port,
        from_tag=sip.new_tag(),
    )


def malformed_catalog(local_ip: str, target_host: str, target_port: int) -> list[MalformedCase]:
    """Return the fixed catalog of controlled malformed requests."""
    cases: list[MalformedCase] = []

    # 1. Missing mandatory Call-ID.
    m = _base_options(local_ip, target_host, target_port)
    m.headers.remove("Call-ID")
    cases.append(MalformedCase(
        "missing-call-id", "OPTIONS with the mandatory Call-ID header removed",
        "server should reject (e.g. 400 Bad Request), not crash", m))

    # 2. CSeq method mismatch.
    m = _base_options(local_ip, target_host, target_port)
    m.headers.set("CSeq", "1 INVITE")
    cases.append(MalformedCase(
        "cseq-method-mismatch", "CSeq method does not match the request method",
        "server should reject with 4xx", m))

    # 3. Via without branch.
    m = _base_options(local_ip, target_host, target_port)
    m.headers.set("Via", f"SIP/2.0/UDP {local_ip}:{target_port}")
    cases.append(MalformedCase(
        "via-no-branch", "top Via missing the required branch parameter",
        "server should reject with 4xx", m))

    # 4. Negative Max-Forwards.
    m = _base_options(local_ip, target_host, target_port)
    m.headers.set("Max-Forwards", "-1")
    cases.append(MalformedCase(
        "negative-max-forwards", "Max-Forwards set to a negative value",
        "server should reject with 4xx", m))

    # 5. Content-Length larger than the (empty) body.
    m = _base_options(local_ip, target_host, target_port)
    m.headers.set("Content-Length", "500")
    cases.append(MalformedCase(
        "content-length-overstated", "Content-Length claims 500 bytes for an empty body",
        "server should not hang waiting for a body; reject or time out cleanly", m))

    # 6. Duplicated From header.
    m = _base_options(local_ip, target_host, target_port)
    m.headers.add("From", "<sip:duplicate@lab.example>;tag=dup")
    cases.append(MalformedCase(
        "duplicate-from", "two From headers present",
        "server should reject with 4xx", m))

    return cases
```

src/siptestwin/security_tests/regression.py

Comparacion de regresion contra un reporte base. *(52 lineas)*

```
"""Regression comparison against an expected baseline report."""

from __future__ import annotations

import json
from dataclasses import dataclass, field
from pathlib import Path


@dataclass
class RegressionResult:
    passed: bool
    differences: list[str] = field(default_factory=list)


def compare_to_baseline(
    current: dict,
    baseline_path: str | Path,
    *,
    tolerate_latency: bool = True,
) -> RegressionResult:
    """Compare a current report dict to a stored baseline JSON report.

```

```
    Compares response-code categories and pass/fail counts, which are the
    stable, repeatable signals. Latency is informational and ignored unless
    ``tolerate_latency`` is False.
    """
    p = Path(baseline_path)
    if not p.is_file():
        return RegressionResult(False, [f"baseline not found: {p}"])
    baseline = json.loads(p.read_text(encoding="utf-8"))

    diffs: list[str] = []
    cur_cat = current.get("metrics", {}).get("response_by_category", {})
    base_cat = baseline.get("metrics", {}).get("response_by_category", {})
    for key in sorted(set(cur_cat) | set(base_cat)):
        if cur_cat.get(key, 0) != base_cat.get(key, 0):
            diffs.append(f"response {key}: baseline={base_cat.get(key, 0)} current={cur_cat.get(key, 0)}")

    cur_sum = current.get("summary", {})
    base_sum = baseline.get("summary", {})
    for key in ("passed", "failed"):
        if cur_sum.get(key) != base_sum.get(key):
            diffs.append(f"summary.{key}: baseline={base_sum.get(key)} current={cur_sum.get(key)}")

    if not tolerate_latency:
        cur_p95 = current.get("metrics", {}).get("latency", {}).get("p95_ms", 0)
        base_p95 = baseline.get("metrics", {}).get("latency", {}).get("p95_ms", 0)
        if base_p95 and cur_p95 > base_p95 * 1.5:
            diffs.append(f"latency p95 regressed: baseline={base_p95} current={cur_p95}")

    return RegressionResult(passed=not diffs, differences=diffs)
```

src/siptestwin/testserver.py

Servidor SIP de prueba local (UAS) para validar en loopback sin sistemas externos: OPTIONS, REGISTER+Digest, INVITE/ACK/BYE, y rechazo de malformados. *(197 lineas)*

```
"""A minimal local SIP test server (UAS simulator) for loopback validation.

This is NOT a production SIP server. It implements just enough behaviour to
exercise SIPTestWin end-to-end on localhost without any external system:

* OPTIONS   -> 200 OK (or 405 for a configured unsupported method)
* REGISTER  -> 401 challenge, then 200 OK once a valid Digest response arrives
* INVITE    -> 100 Trying, 180 Ringing, 200 OK (expects ACK)
* BYE       -> 200 OK
* CANCEL    -> 200 OK
* malformed -> 400 Bad Request

It binds to loopback by default. It exists so that tests and demos never need
to point at a third-party server.
"""

from __future__ import annotations

import hashlib
import secrets
import threading
import time

from . import sip_message as sip
from .authentication.digest import _parse_params
from .sip_message import SIPMessage
from .transport.tcp import TCPTransport
from .transport.udp import UDPTransport

REALM = "siptestwin.lab"


class TestSIPServer:
    def __init__(
        self,
        host: str = "127.0.0.1",
        port: int = 0,
        *,
        transport: str = "udp",
        require_auth: bool = False,
        username: str = "tester",
        password: str = "labpass",
        unsupported_methods: tuple[str, ...] = (),
    ) -> None:
        self.host = host
```

```
        self.require_auth = require_auth
        self.username = username
        self.password = password
        self.unsupported = {m.upper() for m in unsupported_methods}
        self._nonce = secrets.token_hex(16)
        self.received: list[SIPMessage] = []
        self._lock = threading.Lock()

        if transport.lower() == "udp":
            self._transport = UDPTransport(bind_host=host, bind_port=port)
        else:
            self._transport = TCPTransport(bind=(host, port), listen=True)
        self._transport_name = transport.upper()

    # ----- lifecycle -----------------------------------------------------
    def start(self) -> "TestSIPServer":
        self._transport.start(self._on_message)
        return self

    @property
    def port(self) -> int:
        return self._transport.local_addr()[1]

    def stop(self) -> None:
        self._transport.close()

    def __enter__(self) -> "TestSIPServer":
        return self.start()

    def __exit__(self, *exc) -> None:
        self.stop()

    # ----- request handling ----------------------------------------------
    def _on_message(self, data: bytes, addr: tuple[str, int]) -> None:
        try:
            msg = sip.parse(data, strict=False)
        except sip.SIPParseError:
            return
        if not msg.is_request:
            return
        with self._lock:
            self.received.append(msg)

        resp = self._handle(msg)
        for r in resp:
            self._transport.send(r.serialize(), addr)

    def _handle(self, req: SIPMessage) -> list[SIPMessage]:
        problem = self._protocol_problem(req)
        if problem:
            return [sip.build_response(req, 400, problem)]

        method = (req.method or "").upper()
        if method in self.unsupported:
            return [sip.build_response(req, 405, "Method Not Allowed",
                                       extra_headers={"Allow": "INVITE, ACK, BYE, CANCEL, OPTIONS"})]
        if method == "OPTIONS":
            return [sip.build_response(req, 200, extra_headers={
                "Allow": "INVITE, ACK, CANCEL, BYE, OPTIONS, REGISTER",
                "Accept": "application/sdp",
            })]
        if method == "REGISTER":
            return self._handle_auth(req, 401, "WWW-Authenticate")
        if method == "INVITE":
            if self.require_auth:
                auth = self._handle_auth(req, 407, "Proxy-Authenticate")
                if auth[0].status_code != 200:
                    return auth
            return self._invite_responses(req)
        if method in ("ACK",):
            return []  # ACK is not answered
        if method == "BYE":
            return [sip.build_response(req, 200, to_tag=sip.new_tag())]
        if method == "CANCEL":
            return [sip.build_response(req, 200)]
        return [sip.build_response(req, 501, "Not Implemented")]

    def _invite_responses(self, req: SIPMessage) -> list[SIPMessage]:
        to_tag = sip.new_tag()
        contact = f"<sip:{self.username}@{self.host}:{self.port}>"
        sdp = sip.generate_sdp(self.host, port=40000)
        return [
            sip.build_response(req, 100),
            sip.build_response(req, 180, to_tag=to_tag),
```

```
            sip.build_response(req, 200, to_tag=to_tag, contact=contact,
                               extra_headers={"Content-Type": "application/sdp"}, body=sdp),
        ]

    def _handle_auth(self, req: SIPMessage, challenge_code: int,
                     challenge_header: str) -> list[SIPMessage]:
        resp_header = "Authorization" if challenge_header == "WWW-Authenticate" else "Proxy-Authorization"
        auth = req.headers.get(resp_header)
        if not auth:
            www = (f'Digest realm="{REALM}", nonce="{self._nonce}", '
                   f'algorithm=MD5, qop="auth"')
            return [sip.build_response(req, challenge_code,
                                       to_tag=sip.new_tag(),
                                       extra_headers={challenge_header: www})]
        if self._verify_digest(auth, req.method or ""):
            return [sip.build_response(req, 200, to_tag=sip.new_tag(),
                                       extra_headers={"Expires": "3600"})]
        return [sip.build_response(req, 403, "Forbidden")]

    def _verify_digest(self, auth_value: str, method: str) -> bool:
        scheme, _, rest = auth_value.strip().partition(" ")
        if scheme.lower() != "digest":
            return False
        p = _parse_params(rest)
        if p.get("username") != self.username or p.get("nonce") != self._nonce:
            return False
        ha1 = hashlib.md5(f"{self.username}:{REALM}:{self.password}".encode()).hexdigest()
        ha2 = hashlib.md5(f"{method}:{p.get('uri', '')}".encode()).hexdigest()
        if p.get("qop") == "auth":
            expected = hashlib.md5(
                f"{ha1}:{self._nonce}:{p.get('nc','')}:{p.get('cnonce','')}:auth:{ha2}".encode()
            ).hexdigest()
        else:
            expected = hashlib.md5(f"{ha1}:{self._nonce}:{ha2}".encode()).hexdigest()
        return secrets.compare_digest(expected, p.get("response", ""))

    def _protocol_problem(self, req: SIPMessage) -> str | None:
        for h in ("Call-ID", "From", "To", "CSeq", "Via"):
            if h not in req.headers:
                return f"Missing {h}"
            if len(req.headers.get_all(h)) > 1 and h in ("From", "To", "Call-ID", "CSeq"):
                return f"Duplicate {h}"
        cseq = req.headers.get("CSeq", "")
        parts = cseq.split()
        if len(parts) != 2 or not parts[0].isdigit():
            return "Malformed CSeq"
        if parts[1].upper() != (req.method or "").upper():
            return "CSeq method mismatch"
        via = req.headers.get("Via", "")
        if "branch=" not in via:
            return "Via without branch"
        mf = req.headers.get("Max-Forwards")
        if mf is not None:
            try:
                if int(mf) < 0:
                    return "Negative Max-Forwards"
            except ValueError:
                return "Malformed Max-Forwards"
        return None


def run_background(**kwargs) -> TestSIPServer:
    """Start a server on a background thread and return it (call .stop())."""
    server = TestSIPServer(**kwargs)
    server.start()
    # Give the OS a moment to bind before the caller sends to it.
    time.sleep(0.05)
    return server
```

src/siptestwin/cli.py

Interfaz de linea de comandos (Typer): version, validate-config, validate-scenario, run, report, testserver, interfaces, doctor. *(353 lineas)*

```
"""SIPTestWin command-line interface (Typer).

Commands:
    version           Print version information.
    validate-config   Load and strictly validate a configuration file.
    validate-scenario Parse and validate a scenario XML file.
```

```
    run               Execute a scenario (simulate by default; --execute to send).
    report            Summarize previously written JSON reports.
    testserver        Run the local loopback test SIP server.
    doctor            Diagnose the local environment (ports, imports, firewall).

Safety: ``run`` operates in SIMULATE mode unless ``--execute`` is given. An
active run requires the target to be on security.authorized_targets and prompts
for confirmation unless ``--yes`` is passed.
"""

from __future__ import annotations

import signal
import socket
import sys
import time
from pathlib import Path

import typer
from rich.console import Console
from rich.table import Table

from . import __version__
from . import config as config_mod
from . import reporting, scenario_parser
from .capture import SipTracer
from .engine import Engine
from .netutils import list_local_ipv4
from .safety import (
    AuditLog,
    AuthorizationError,
    RunLimits,
    confirmation_prompt,
    ensure_authorized,
    redact_config,
)

app = typer.Typer(
    add_completion=False,
    help="SIPTestWin - authorized SIP interoperability / security testing for Windows.",
    no_args_is_help=True,
)
console = Console()
err_console = Console(stderr=True)


# ---------------------------------------------------------------------------
# helpers
# ---------------------------------------------------------------------------

def _load_config_or_exit(path: Path, overrides: dict) -> config_mod.Config:
    try:
        cfg = config_mod.load(path)
    except config_mod.ConfigError as exc:
        err_console.print(f"[red]config error:[/red] {exc}")
        raise typer.Exit(2)
    _apply_overrides(cfg, overrides)
    errors = cfg.validate()
    if errors:
        err_console.print("[red]invalid configuration:[/red]")
        for e in errors:
            err_console.print(f"  - {e}")
        raise typer.Exit(2)
    return cfg


def _apply_overrides(cfg: config_mod.Config, overrides: dict) -> None:
    if overrides.get("host"):
        cfg.target.host = overrides["host"]
    if overrides.get("port"):
        cfg.target.port = overrides["port"]
    if overrides.get("transport"):
        cfg.target.transport = overrides["transport"]
    if overrides.get("scenario"):
        cfg.execution.scenario = overrides["scenario"]
    if overrides.get("calls"):
        cfg.execution.calls = overrides["calls"]
    if overrides.get("concurrency"):
        cfg.execution.concurrency = overrides["concurrency"]
    if overrides.get("call_rate"):
        cfg.execution.call_rate = overrides["call_rate"]
    if overrides.get("output_dir"):
        cfg.reporting.output_dir = overrides["output_dir"]
    if overrides.get("local_host"):
```

```
        cfg.execution.local_host = overrides["local_host"]
    if overrides.get("authorize"):
        for host in overrides["authorize"]:
            if host not in cfg.security.authorized_targets:
                cfg.security.authorized_targets.append(host)


def _build_tracer(cfg, capture, pcap, trace_console, trace_full, trace_file):
    """Construct a SipTracer from config + CLI flags, or None if tracing off."""
    cap = cfg.capture
    cli_requested = capture or pcap or trace_console
    if not (cap.enabled or cli_requested):
        return None

    console_on = trace_console or (cap.enabled and cap.console)
    console_full = trace_full or (cap.enabled and cap.console_full)
    text_on = capture or (cap.enabled and cap.text)
    pcap_on = pcap or (cap.enabled and cap.pcap)
    # If tracing is on but no sink resolved (e.g. bare config enabled), default to text.
    if not (console_on or text_on or pcap_on):
        text_on = True

    base = trace_file or cap.file or \
        str(Path(cfg.reporting.output_dir) / f"trace-{time.strftime('%Y%m%d-%H%M%S')}")
    text_path = f"{base}.trace.log" if text_on else None
    pcap_path = f"{base}.pcap" if pcap_on else None
    return SipTracer(
        to_console=console_on,
        text_path=text_path,
        pcap_path=pcap_path,
        console_full=console_full,
        console_print=lambda m: console.print(m, highlight=False, soft_wrap=True),
    )


# ---------------------------------------------------------------------------
# commands
# ---------------------------------------------------------------------------

@app.command()
def version() -> None:
    """Print version information."""
    console.print(f"SIPTestWin {__version__}  (python {sys.version.split()[0]})")


@app.command("validate-config")
def validate_config(
    config: Path = typer.Option(..., "--config", "-c", exists=True, help="YAML/JSON config file."),
) -> None:
    """Load and strictly validate a configuration file."""
    cfg = _load_config_or_exit(config, {})
    console.print("[green]OK[/green] configuration is valid.")
    console.print_json(data=redact_config(cfg))


@app.command("validate-scenario")
def validate_scenario(
    file: Path = typer.Option(..., "--file", "-f", exists=True, help="Scenario XML file."),
) -> None:
    """Parse and validate a scenario XML file."""
    try:
        scenario = scenario_parser.parse_file(file)
    except scenario_parser.ScenarioError as exc:
        err_console.print(f"[red]scenario error:[/red] {exc}")
        raise typer.Exit(2)
    console.print(f"[green]OK[/green] scenario '{scenario.name}' is valid.")
    console.print(f"  flow: {scenario.summary()}")


@app.command()
def run(
    config: Path = typer.Option(..., "--config", "-c", exists=True, help="YAML/JSON config file."),
    execute: bool = typer.Option(False, "--execute", help="Send real packets (default is simulate)."),
    yes: bool = typer.Option(False, "--yes", "-y", help="Skip the active-run confirmation prompt."),
    host: str = typer.Option(None, "--host", help="Override target host."),
    port: int = typer.Option(None, "--port", help="Override target port."),
    transport: str = typer.Option(None, "--transport", help="Override transport (udp/tcp/tls)."),
    scenario: str = typer.Option(None, "--scenario", help="Override scenario file path."),
    calls: int = typer.Option(None, "--calls", help="Override number of calls."),
    concurrency: int = typer.Option(None, "--concurrency", help="Override concurrency."),
    call_rate: float = typer.Option(None, "--call-rate", help="Override call rate (calls/s)."),
    local_host: str = typer.Option(None, "--local-host",
                                   help="Source IP for Via/Contact/bind (see 'interfaces')."),
    authorize: list[str] = typer.Option(None, "--authorize",
```

```
                                       help="Add a host/CIDR to the authorized-targets allow-list."),
    output_dir: str = typer.Option(None, "--output-dir", help="Override report output directory."),
    capture: bool = typer.Option(False, "--capture", help="Trace SIP messages to a .trace.log file."),
    pcap: bool = typer.Option(False, "--pcap", help="Also write a Wireshark-openable .pcap."),
    trace_console: bool = typer.Option(False, "--trace-console", help="Print each SIP message line to the conso
le."),
    trace_full: bool = typer.Option(False, "--trace-full", help="With --trace-console, print full messages."),
    trace_file: str = typer.Option(None, "--trace-file", help="Base path for trace/pcap output."),
) -> None:
    """Execute a scenario. SIMULATE by default; use --execute to transmit."""
    overrides = {
        "host": host, "port": port, "transport": transport, "scenario": scenario,
        "calls": calls, "concurrency": concurrency, "call_rate": call_rate,
        "output_dir": output_dir, "authorize": authorize, "local_host": local_host,
    }
    cfg = _load_config_or_exit(config, overrides)

    try:
        parsed = scenario_parser.parse_file(cfg.execution.scenario)
    except scenario_parser.ScenarioError as exc:
        err_console.print(f"[red]scenario error:[/red] {exc}")
        raise typer.Exit(2)

    simulate = not execute
    limits = RunLimits.from_config(cfg)

    if not simulate:
        try:
            ensure_authorized(cfg)
        except AuthorizationError as exc:
            err_console.print(f"[red]authorization error:[/red] {exc}")
            raise typer.Exit(3)
        console.print(confirmation_prompt(cfg, limits))
        if not yes:
            if not typer.confirm("Proceed with this ACTIVE test?", default=False):
                console.print("aborted.")
                raise typer.Exit(1)

    tracer = _build_tracer(cfg, capture, pcap, trace_console, trace_full, trace_file)
    engine = Engine(cfg, parsed, simulate=simulate, tracer=tracer,
                    on_log=lambda m: console.print(f"[dim]{m}[/dim]"))

    # Ctrl+C -> graceful stop.
    def _sigint(_signum, _frame):
        err_console.print("\n[yellow]stopping (Ctrl+C)...[/yellow]")
        engine.stop()
    signal.signal(signal.SIGINT, _sigint)

    audit = AuditLog(cfg.reporting.output_dir)
    if not simulate:
        audit.record("run_start", cfg, {"limits": vars(limits)})

    mode = "SIMULATE (no packets)" if simulate else "ACTIVE"
    console.rule(f"SIPTestWin run - {mode}")
    result = engine.run()
    if tracer is not None:
        tracer.close()
    result.config_summary = redact_config(cfg)

    reporting.print_summary(result, console)
    written = reporting.write_reports(result, cfg.reporting.output_dir, cfg.reporting.formats)
    for p in written:
        console.print(f"[green]report:[/green] {p}")
    if tracer is not None:
        for p in tracer.paths():
            console.print(f"[green]trace:[/green] {p}")
    if not simulate:
        audit.record("run_end", cfg, {
            "passed": result.passed, "failed": result.failed,
            "duration_s": result.duration_s,
        })

    raise typer.Exit(0 if result.failed == 0 else 4)


@app.command()
def report(
    input: Path = typer.Option(..., "--input", "-i", exists=True, help="Directory of JSON reports."),
) -> None:
    """Summarize previously written JSON reports."""
    reports = reporting.load_reports(input)
    if not reports:
        err_console.print("[yellow]no report-*.json files found[/yellow]")
        raise typer.Exit(1)
```

```
    for r in reports:
        s = r.get("summary", {})
        console.print(f"[bold]{r.get('scenario')}[/bold] vs {r.get('target')} "
                      f"({r.get('mode')}) -> passed={s.get('passed')} failed={s.get('failed')} "
                      f"[{r.get('started_at')}]")


@app.command()
def testserver(
    host: str = typer.Option("127.0.0.1", help="Bind host (loopback by default)."),
    port: int = typer.Option(5060, help="Bind port."),
    transport: str = typer.Option("udp", help="udp or tcp."),
    require_auth: bool = typer.Option(False, "--require-auth", help="Require Digest auth."),
) -> None:
    """Run the local loopback test SIP server (for lab validation)."""
    from .testserver import TestSIPServer
    server = TestSIPServer(host=host, port=port, transport=transport, require_auth=require_auth)
    server.start()
    console.print(f"[green]test server listening[/green] on {host}:{server.port}/{transport} "
                  f"(auth={'on' if require_auth else 'off'}); Ctrl+C to stop")
    try:
        signal.pause() if hasattr(signal, "pause") else _wait_forever()
    except KeyboardInterrupt:
        pass
    finally:
        server.stop()
        console.print("stopped.")


def _wait_forever() -> None:
    import time
    while True:
        time.sleep(1)


@app.command()
def interfaces() -> None:
    """List local IPv4 addresses available to trace / bind as source IP."""
    addrs = list_local_ipv4()
    table = Table(title="Local IPv4 addresses", header_style="bold")
    table.add_column("IP")
    table.add_column("Type")
    for a in addrs:
        table.add_row(a.ip, a.label)
    console.print(table)
    console.print("Pick one with:  run --local-host <IP>   "
                  "(or set execution.local_host in the config).")


@app.command()
def doctor() -> None:
    """Diagnose the local environment for SIP testing on Windows."""
    ok = True
    console.rule("SIPTestWin doctor")

    pyver = sys.version_info
    if pyver >= (3, 12):
        console.print(f"[green]OK[/green] Python {pyver.major}.{pyver.minor}.{pyver.micro}")
    else:
        ok = False
        console.print(f"[red]FAIL[/red] Python >= 3.12 required, found {pyver.major}.{pyver.minor}")

    for mod in ("yaml", "typer", "rich"):
        try:
            __import__(mod)
            console.print(f"[green]OK[/green] import {mod}")
        except ImportError:
            ok = False
            console.print(f"[red]FAIL[/red] cannot import {mod}")

    # Can we bind a UDP socket (needed to receive responses)?
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        s.bind(("127.0.0.1", 0))
        bound = s.getsockname()[1]
        s.close()
        console.print(f"[green]OK[/green] UDP bind works (ephemeral port {bound})")
    except OSError as exc:
        ok = False
        console.print(f"[red]FAIL[/red] UDP bind failed: {exc}")

    console.print()
    console.print("[bold]Windows firewall note:[/bold] inbound SIP responses may be blocked by "
                  "Windows Defender Firewall. If responses time out, allow Python/siptestwin.exe "
```

```
342 |                   "for the active network profile, or test against the local testserver first.")
343 |     console.print("[bold]Ports:[/bold] SIP commonly uses UDP/TCP 5060 and TLS 5061. Binding low "
344 |                   "ports is not required for a client; the OS assigns an ephemeral source port.")
345 |     raise typer.Exit(0 if ok else 1)
346 |
347 |
348 | def main() -> None:
349 |     app()
350 |
351 |
352 | if __name__ == "__main__":
353 |     main()
```

src/siptestwin/__main__.py

Punto de entrada para 'python -m siptestwin' y el ejecutable. *(11 lineas)*

```
  1 | """Entry point for ``python -m siptestwin`` and the PyInstaller executable.
  2 |
  3 | An absolute import is used (not ``from .cli``) so the same module works both
  4 | when run as a package module (``python -m siptestwin``) and when PyInstaller
  5 | executes it as a top-level script with no package context.
  6 | """
  7 |
  8 | from siptestwin.cli import main
  9 |
 10 | if __name__ == "__main__":
 11 |     main()
```